Welcome to the world of simulators!!!

Lab 9 · ChampSim and caches

Buffer Slide

- Doubts from last week's lab? Ask them now.
- Outlab: status and doubts (TA update).
- The tasks are not graded. Do them anyway: the lab exam builds on them.
- The last hour is the lab exam.

Buffer Slide

- Outlab 7 + 8 is coming soon.
- Ask and answer more questions today: we have too many chocolates.



Plan for this lab

- Part 1: ChampSim, a cache simulator
- Part 2: The cache hierarchy: how a request travels
- Part 3: Replacement policies
- Part 4: ChampSim under the hood → Tasks 1–3
- Lab exam (last hour)

Part 1

ChampSim

- What it is
- Traces
- Its folders
- Building and configuring

ChampSim

- ChampSim is a simulator of a CPU core and its caches.
- Architects use simulators to test an idea, such as a new cache policy, without building a chip.
- ChampSim runs on Linux and needs only `g++`, `make` and `python3`.
- To install it at home: `git clone --branch 2022-01 https://github.com/ChampSim/ChampSim.git`

ChampSim replays traces

```
$ ls /opt/CS231_Traces
602.gcc_s-2226B.champsimtrace.xz    605.mcf_s-994B.champsimtrace.xz
603.bwaves_s-891B.champsimtrace.xz  soplex_66B.trace.xz
```

- ChampSim does not run your program. It replays a **trace**: the list of instructions a real program executed, with the addresses each one read or wrote.
- The trace never changes, so any change in the result comes from the processor you configured.
- We use four SPEC CPU programs. The lab machines keep them in `/opt/CS231_Traces`.
- The lab makefile finds a trace by name: `trace=soplex` picks `*soplex*.xz`.

Exploring ChampSim: Building

```
Champsim/
├── champsim_config.json
├── config.sh
├── Makefile                (generated)
├── inc/
│   └── cache.h
├── src/
│   ├── cache.cc
│   └── main.cc
├── replacement/
│   ├── lru/lru.cc
│   └── srrip/srrip.cc
├── prefetcher/
├── branch/
└── bin/
    └── champsim            (built)
```

- If you built before, run `make clean` first.

- **Step 1:** `./config.sh champsim_config.json` reads the JSON and writes the `Makefile`.
- **Step 2:** `make` compiles the simulator into `bin/`.

This is the canonical way to build ChampSim. Run both steps again every time you change the JSON.

Exploring ChampSim: The configuration

```
Champsim/
├── champsim_config.json
├── config.sh
├── Makefile              (generated)
├── inc/
│   └── cache.h
├── src/
│   ├── cache.cc
│   └── main.cc
├── replacement/
│   ├── lru/lru.cc
│   └── srrip/srrip.cc
├── prefetcher/
├── branch/
└── bin/
    └── champsim          (built)
```

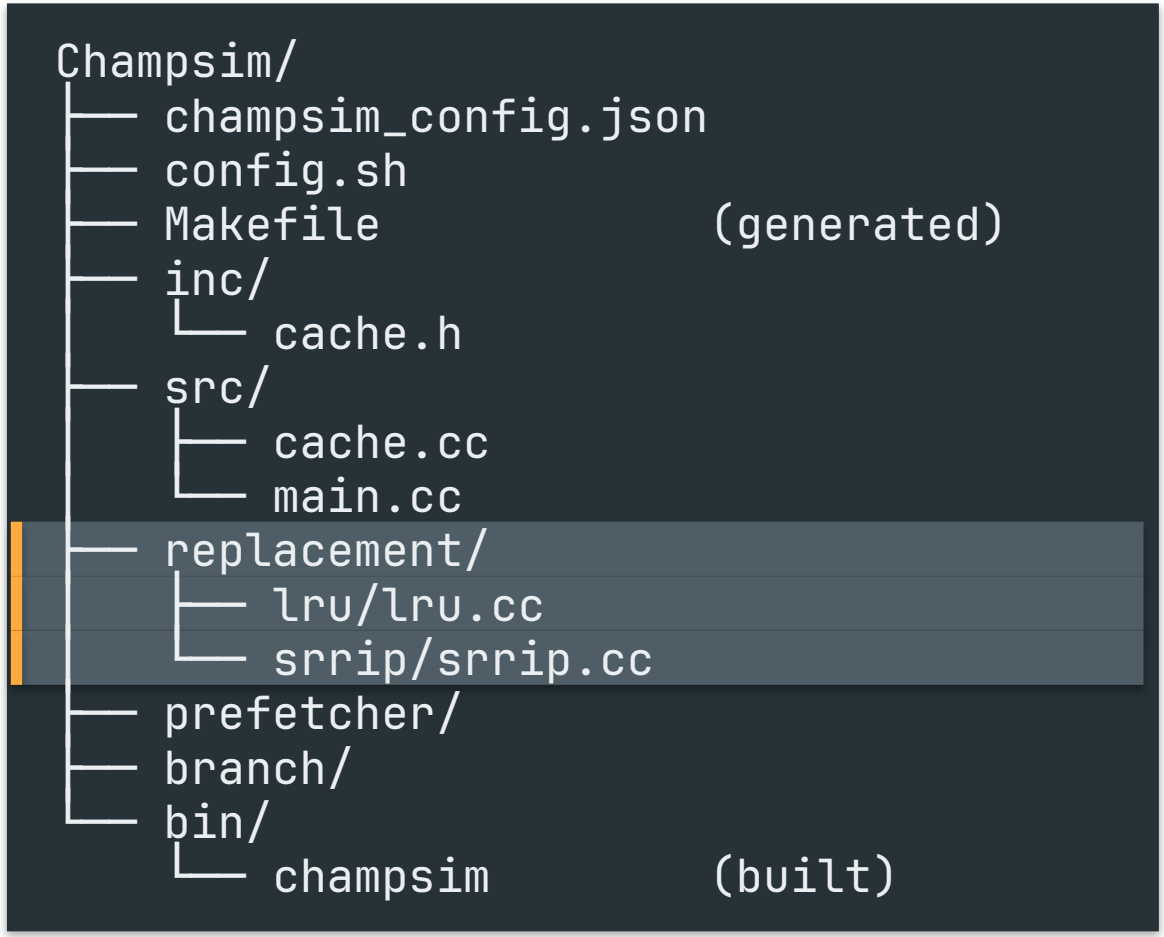
- `champsim_config.json` describes the processor you simulate.
- You set cache sizes, latencies, queue sizes and policies here.

Exploring ChampSim: The simulator code

```
Champsim/
├── champsim_config.json
├── config.sh
├── Makefile                (generated)
├── inc/
│   └── cache.h
├── src/
│   ├── cache.cc
│   └── main.cc
├── replacement/
│   ├── lru/lru.cc
│   └── srrip/srrip.cc
├── prefetcher/
├── branch/
└── bin/
    └── champsim            (built)
```

- `src/cache.cc` decides what a cache does with each request.
- `src/main.cc` holds the main loop and prints the statistics.
- `inc/cache.h` declares the `CACHE` class.

Exploring ChampSim: Replacement policies



- `replacement/` has one folder per replacement policy.
- The JSON names the folder that each cache uses.
- We explain replacement policies in Part 3.

Exploring ChampSim: The binary

```
Champsim/
├── champsim_config.json
├── config.sh
├── Makefile                (generated)
├── inc/
│   └── cache.h
├── src/
│   ├── cache.cc
│   └── main.cc
├── replacement/
│   ├── lru/lru.cc
│   └── srrip/srrip.cc
├── prefetcher/
├── branch/
└── bin/
    └── champsim            (built)
```

- `make` writes the simulator to `bin/`.
- The file name comes from `executable_name` in the JSON.

The whole configuration file

```
{
  "executable_name": "bin/champsim",
  "block_size": 64,
  "num_cores": 1,
  "ooo_cpu": [ { ..., "branch_predictor": "bimodal" } ],
  "L1I":  { "sets": 64,   "ways": 8,  ..., "replacement": "lru" },
  "L1D":  { "sets": 64,   "ways": 12, ..., "replacement": "lru" },
  "L2C":  { "sets": 1024, "ways": 8,  ..., "replacement": "lru" },
  "ITLB": { ... },  "DTLB": { ... },  "STLB": { ... },
  "LLC":  { "sets": 2048, "ways": 16, ..., "replacement": "lru" },
  "physical_memory": { ... },
  "virtual_memory": { ... }
}
```

- The file has one block per part of the processor. We only touch the cache blocks.
- Each cache block can name its own `replacement` policy. A block without that line uses `lru`.
- The given file writes the line only for the `LLC`. For the tasks, the makefile sets it in the `L2C` block.

One cache block

```
"LLC": {
    "sets": 2048,
    "ways": 16,
    "rq_size": 32,
    "wq_size": 32,
    "mshr_size": 64,
    "latency": 20,
    "max_read": 1,
    "max_write": 1,
    "prefetcher": "no",
    "replacement": "lru"
}
```

- `sets` and `ways` give the shape of the cache.
- `latency` is the number of cycles a hit takes.
- `rq_size`, `wq_size` and `mshr_size` are buffer sizes. Part 2 explains them.
- `max_read` and `max_write` limit the requests handled per cycle.
- `replacement` names a folder under `replacement/`.

Running ChampSim

```
$ ./bin/champsim --warmup_instructions 1000000 \
                 --simulation_instructions 10000000 \
                 /opt/CS231_Traces/soplex_66B.trace.xz
```

- The **warm-up** instructions fill the caches. ChampSim throws away their statistics.
- The **simulation** instructions are the ones that count. This part is the region of interest (ROI).
- At the end, the simulation prints its **statistics** for the ROI: the IPC, and the hits and misses of every cache.
- Use **1M + 10M** to check that something works, and **20M + 25M** for real numbers.

We cover the general concepts first, then map them to ChampSim.

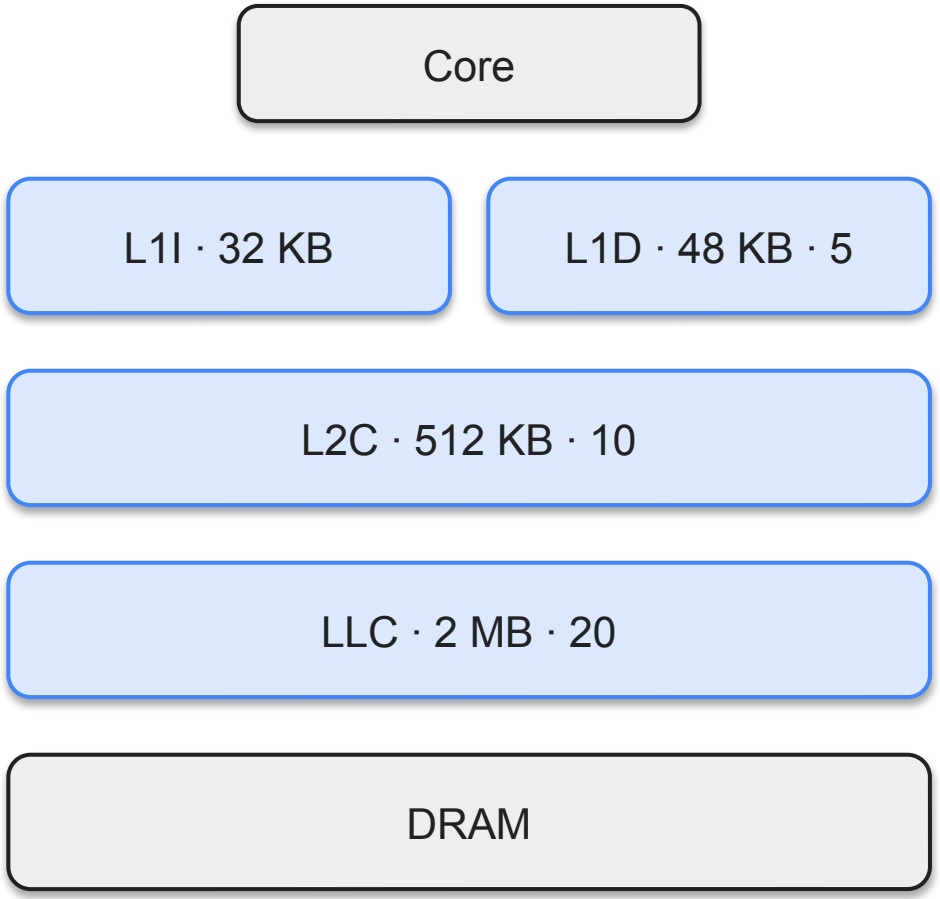
Part 2

The cache hierarchy

- Levels and addresses
- Tracing a load
- Packets and request types
- Tracing a store
- Queues and MSHRs
- Metrics

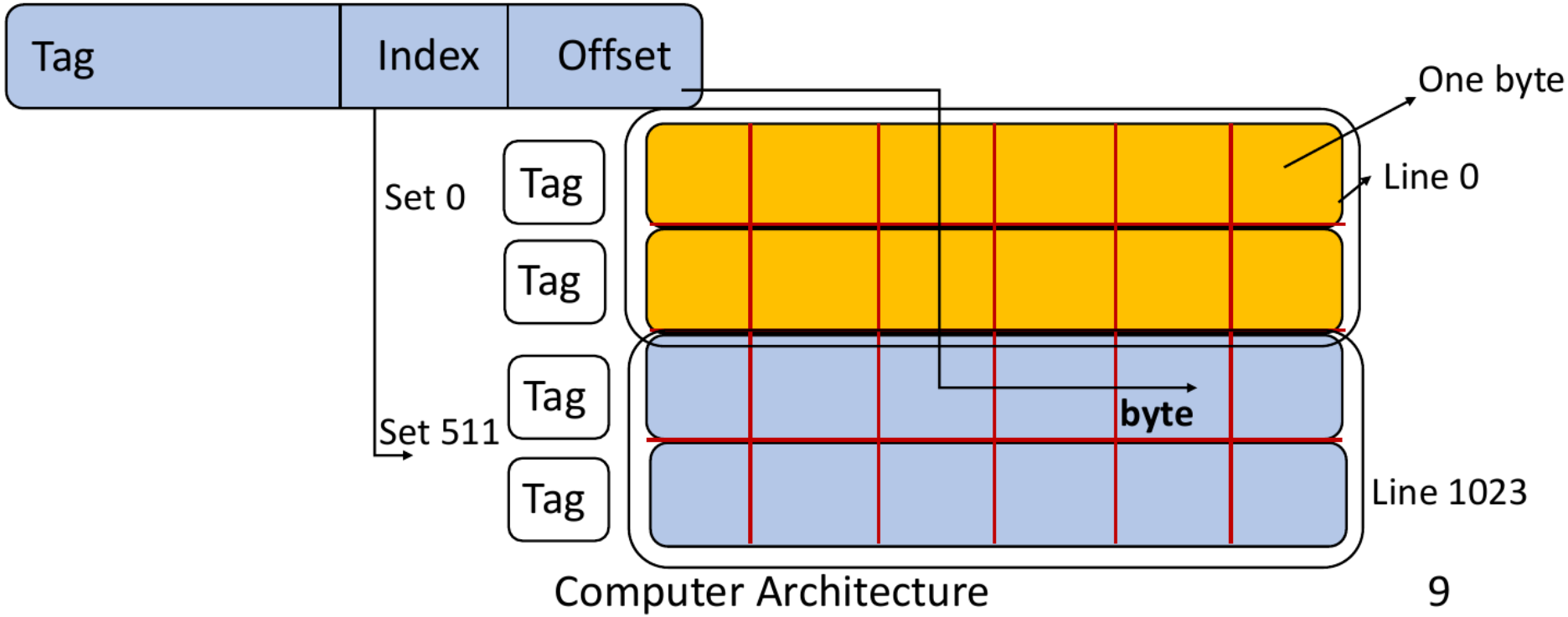
The cache hierarchy

- Each level is bigger and slower than the one above it.
- The L1 is split into the L1I for instructions and the L1D for data.
- When a level misses, it sends a request to the level below.
- The numbers in the boxes are hit latencies in cycles, taken from the JSON.

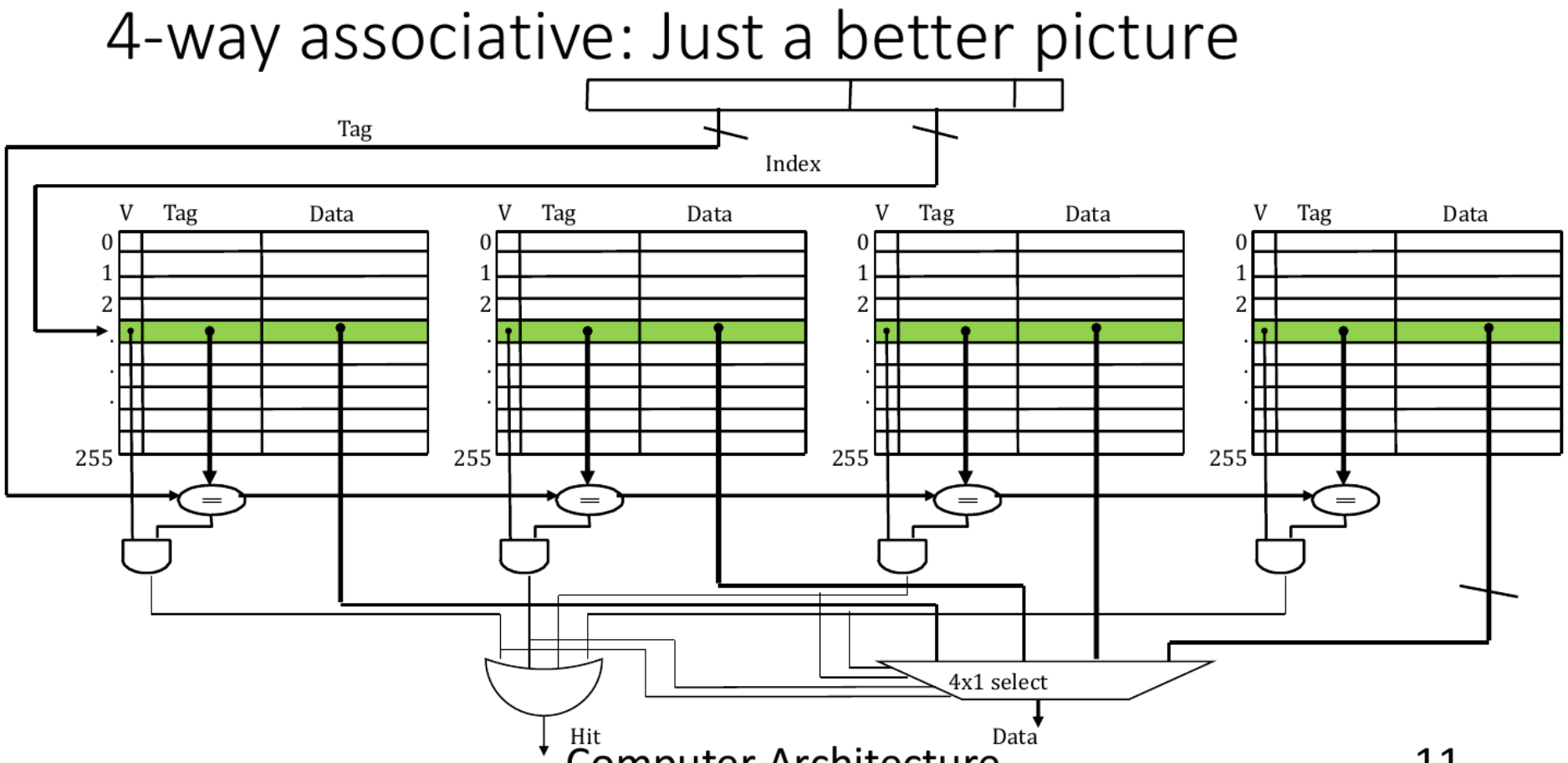


Recap: sets and ways

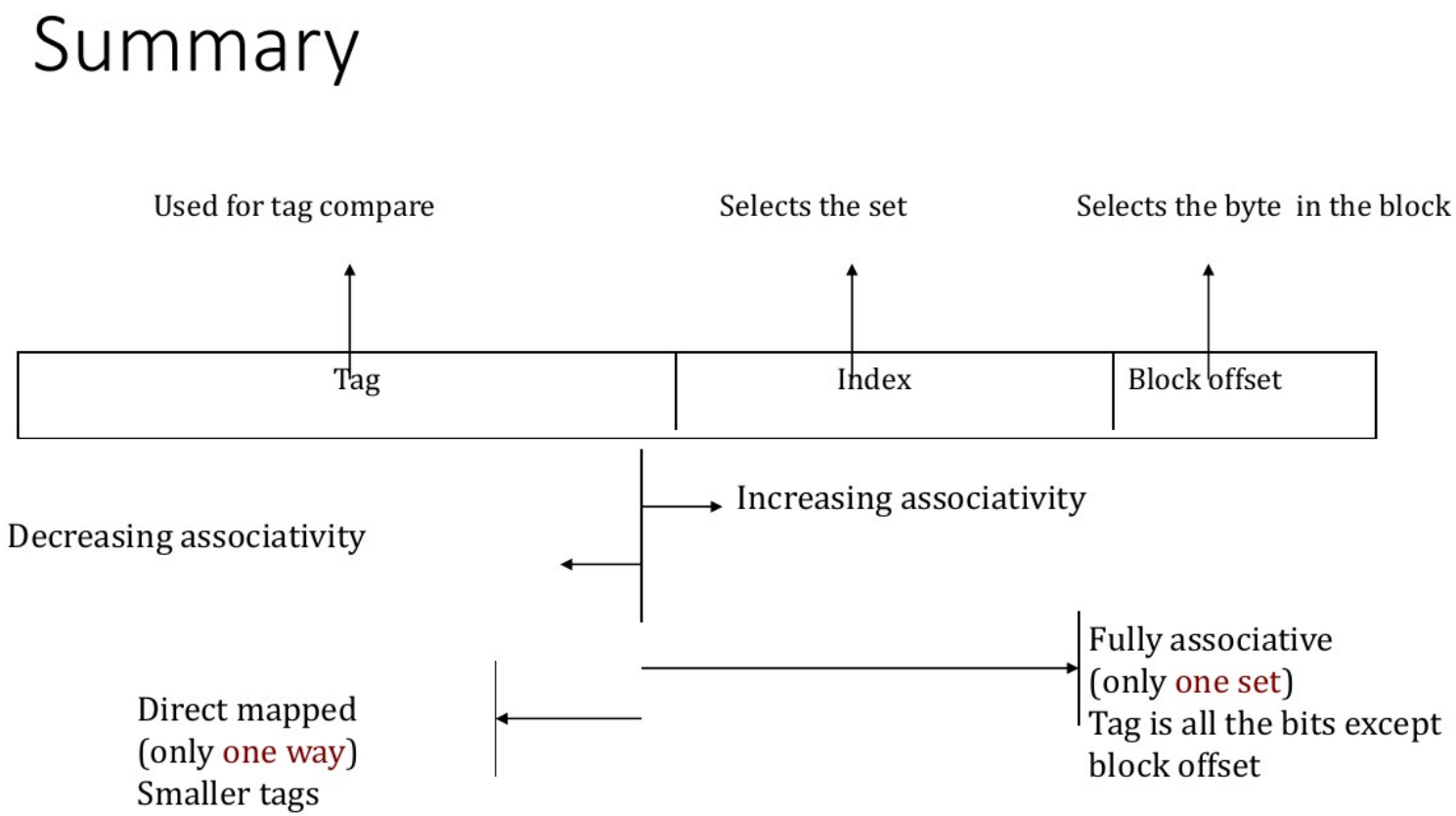
What if we have multiple ways?



Recap: a 4-way set-associative cache



Recap: tag, index, offset



Recap: the bits of an address

- Lines are **64** bytes, so the low **6** bits are the **offset** inside the line.
- The next bits are the **index**. They pick the set: **$\log_2$(sets)** bits.
- The rest of the address is the **tag**. It tells apart the lines that share a set.
- In our configuration:
  - L1D: 48 KB, 12 ways → **64** sets → **6** index bits
  - LLC: 2 MB, 16 ways → **2048** sets → **11** index bits

Practice: find the set

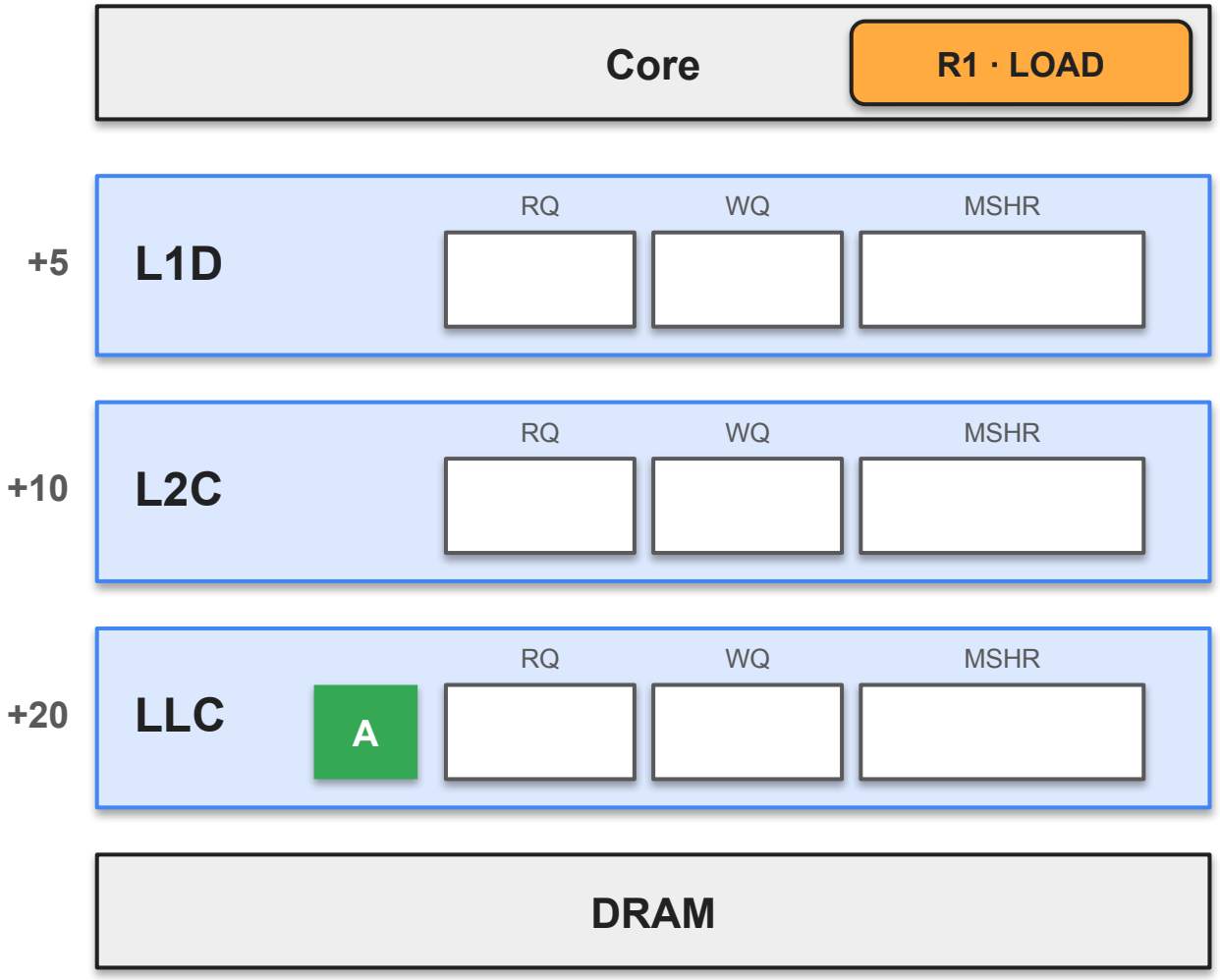
- The core loads from address **0x1234ABC0**.
- Which byte of the line, which set and which tag does it use in the L1D?
- Which set does it use in the LLC?

H: Write the address in binary. Cut off 6 bits for the offset, then the index bits.

Practice: answer

- 0x1234ABC0 = …0100 1010 1011 1100 0000 in binary.
- Offset = low 6 bits = **0**.
- L1D index = next 6 bits = 101111 = **47**. Tag = address >> 12 = **0x1234A**.
- LLC index = next 11 bits = 01010101111 = **687**. Tag = address >> 17 = **0x91A**.

Tracing a demand load

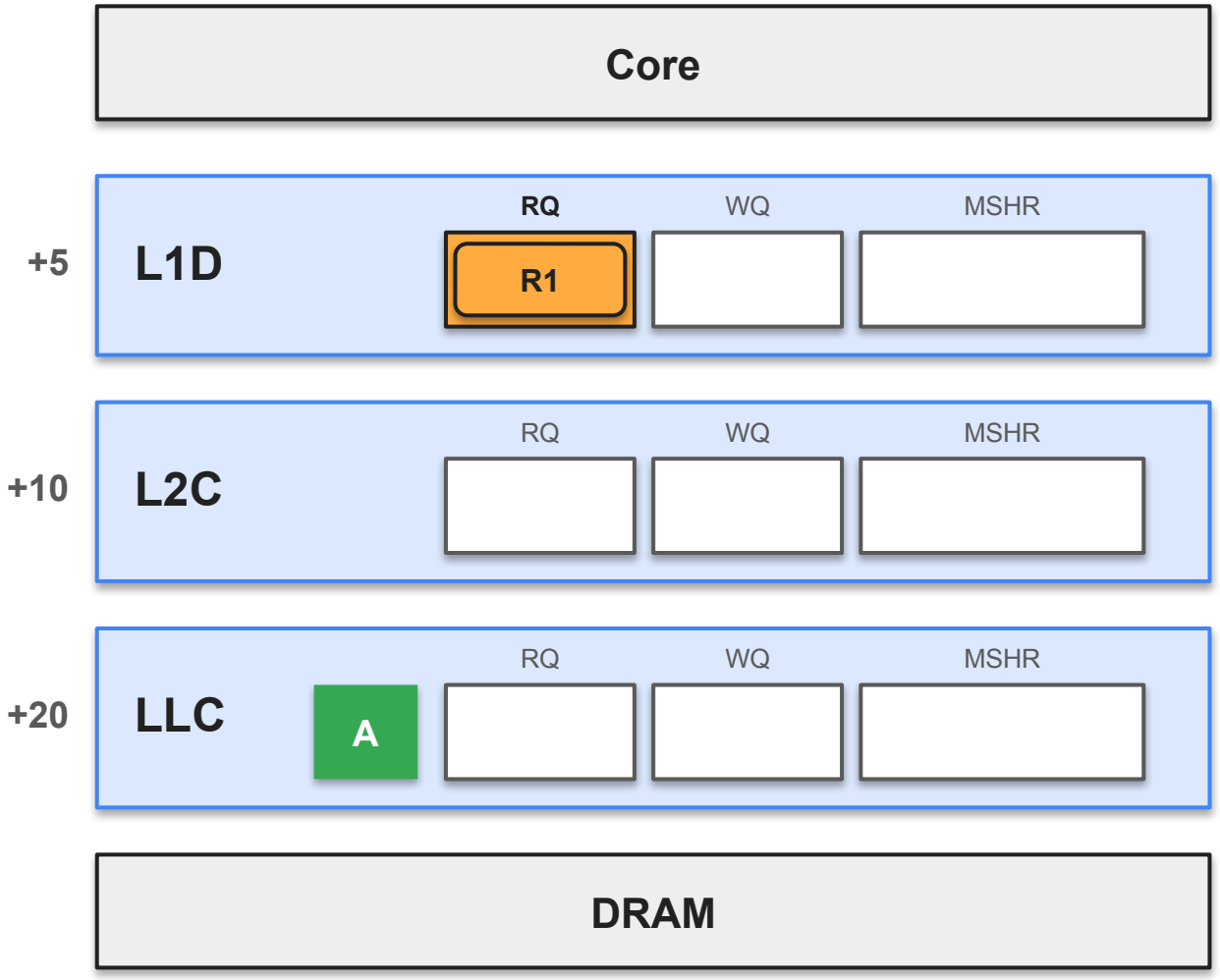


- We follow one load, **R1**, for line A.
- Line A is in the LLC, but not in the L1D or the L2C.
- RQ and MSHR are two kinds of buffer. We explain them later.
- The numbers on the left are hit latencies in cycles.

Q: What does it mean for a line to "reside" in the LLC?

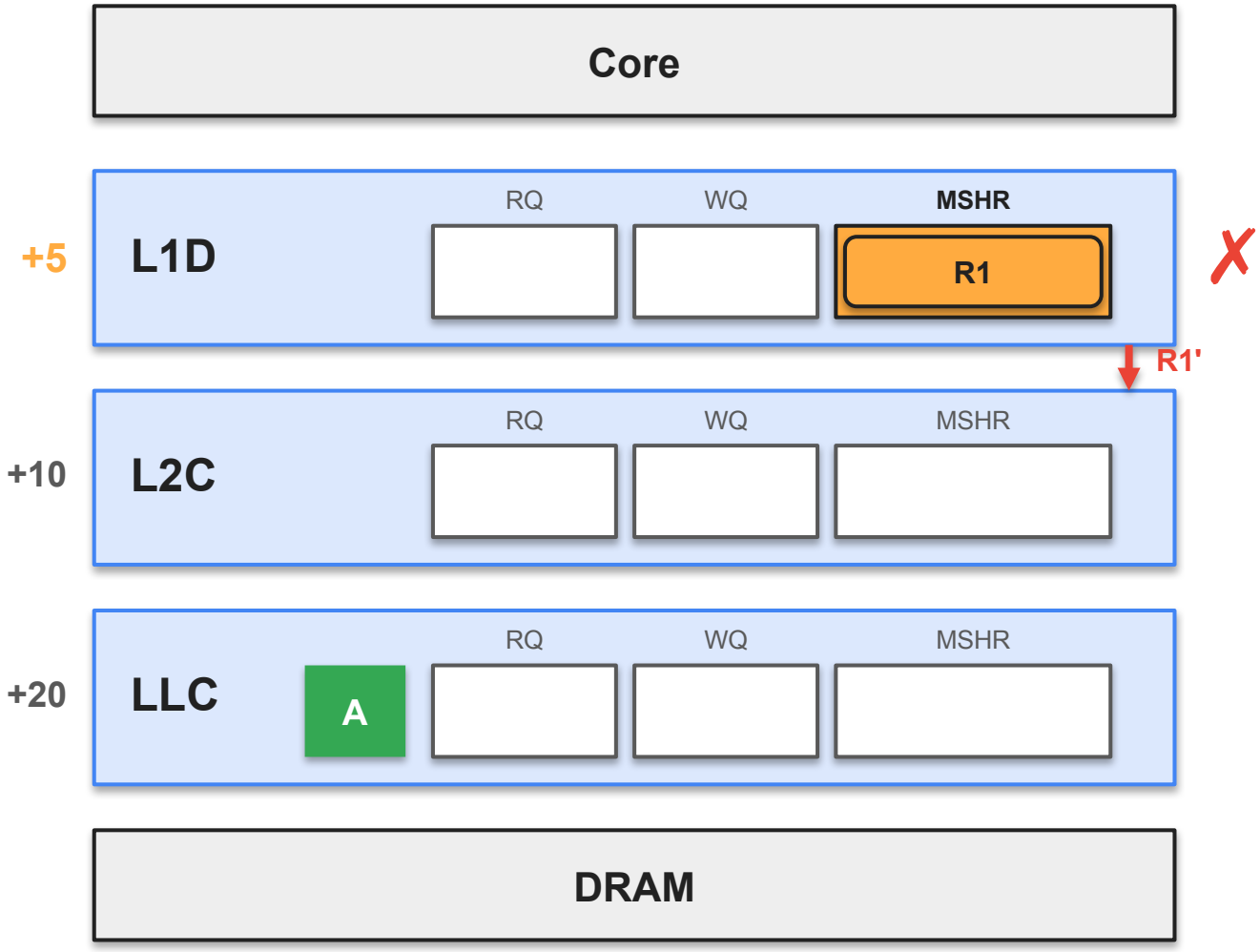
Q: What could each buffer be for?

R1 enters the L1D



- The core sends R1 to the L1D.
- R1 waits in the L1D read queue (RQ).

L1D lookup: miss

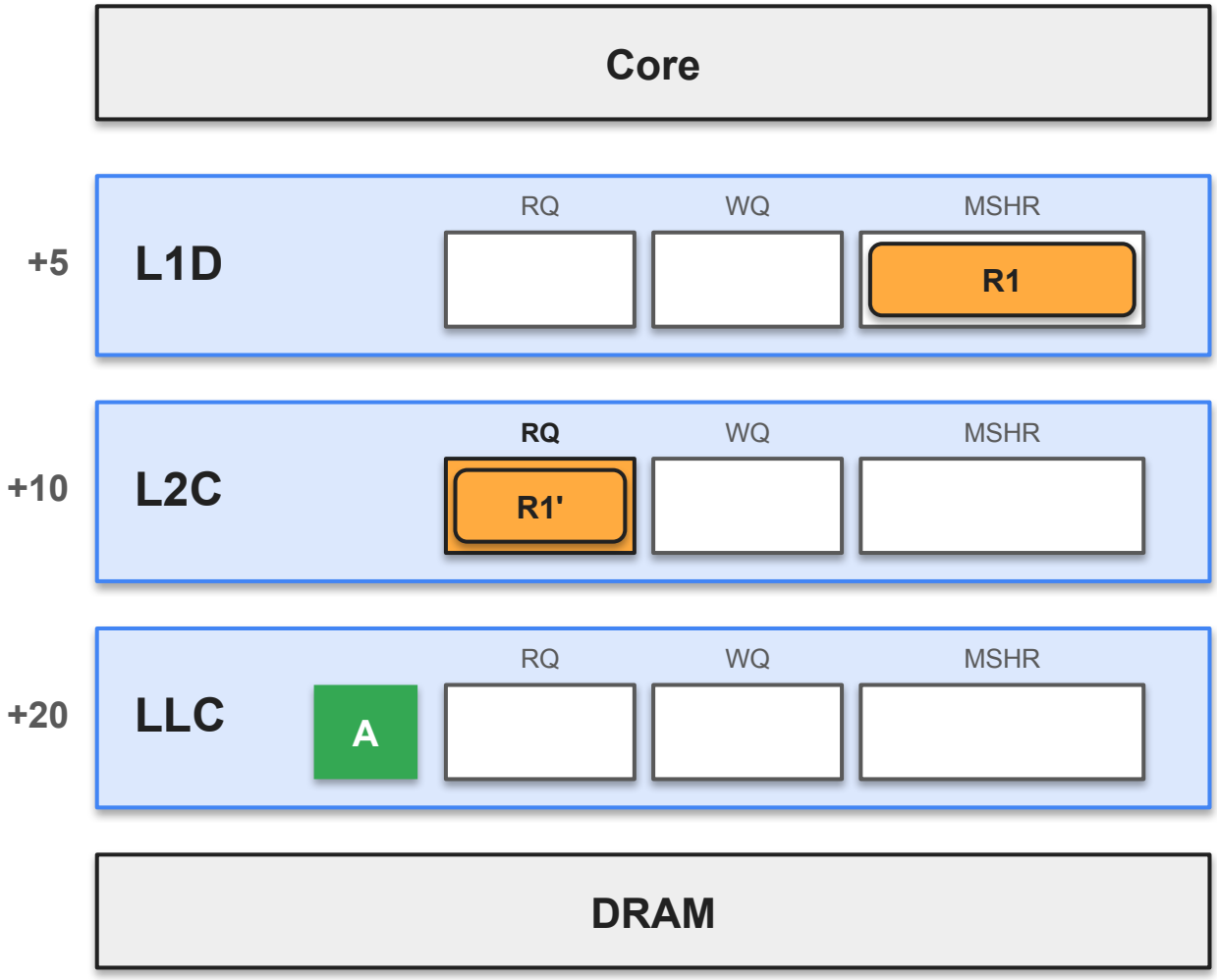




64 sets in the L1D

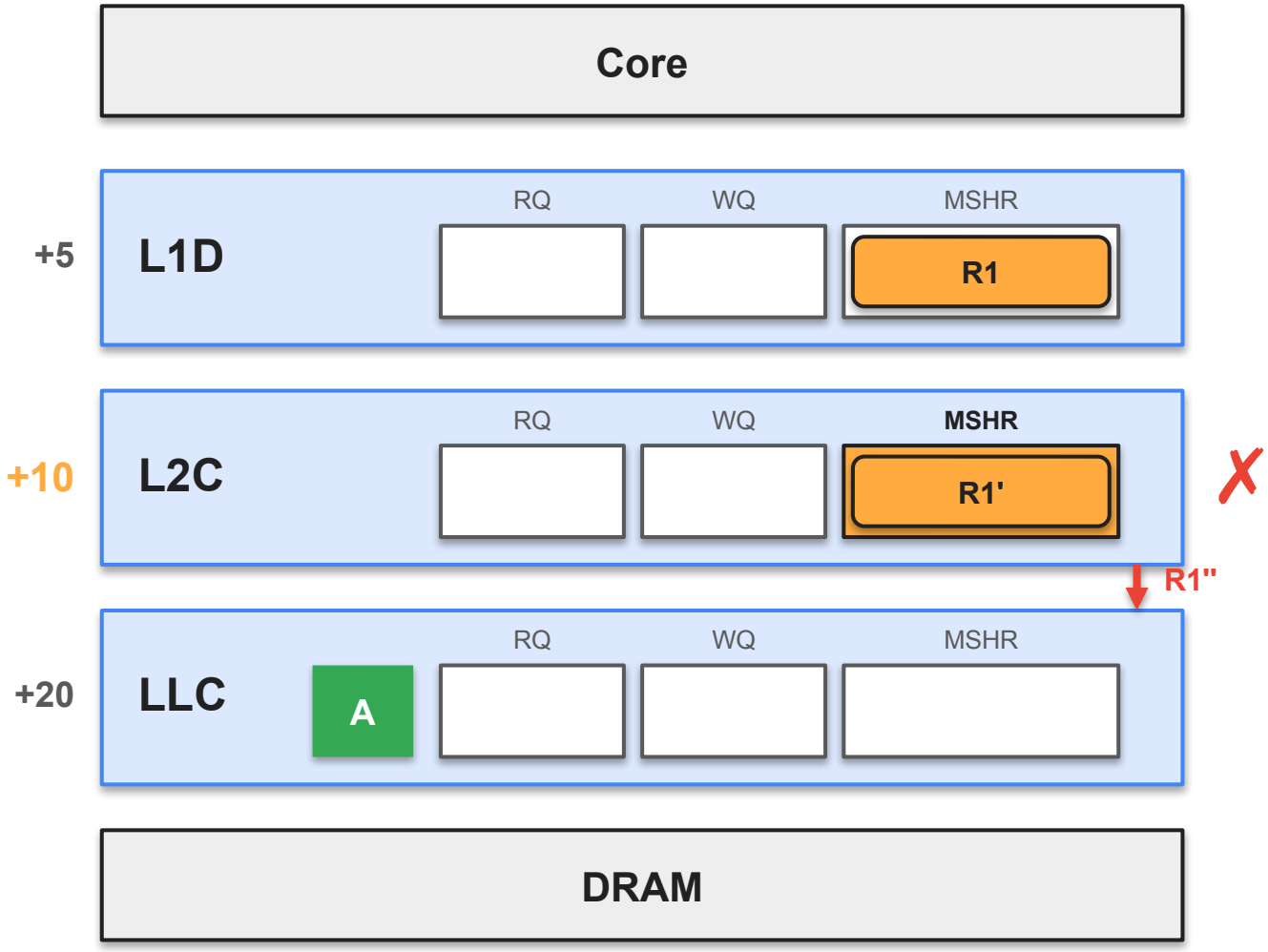
- We read the index bits to find the set.
- Then we compare the tag bits with the tag of every valid way in that set.
- No tag matches, so it is a **miss**!
- R1 moves to an MSHR, and the L1D sends **R1'** down to the L2C.

R1' enters the L2C



- R1' waits in the L2C read queue.

L2C lookup: miss

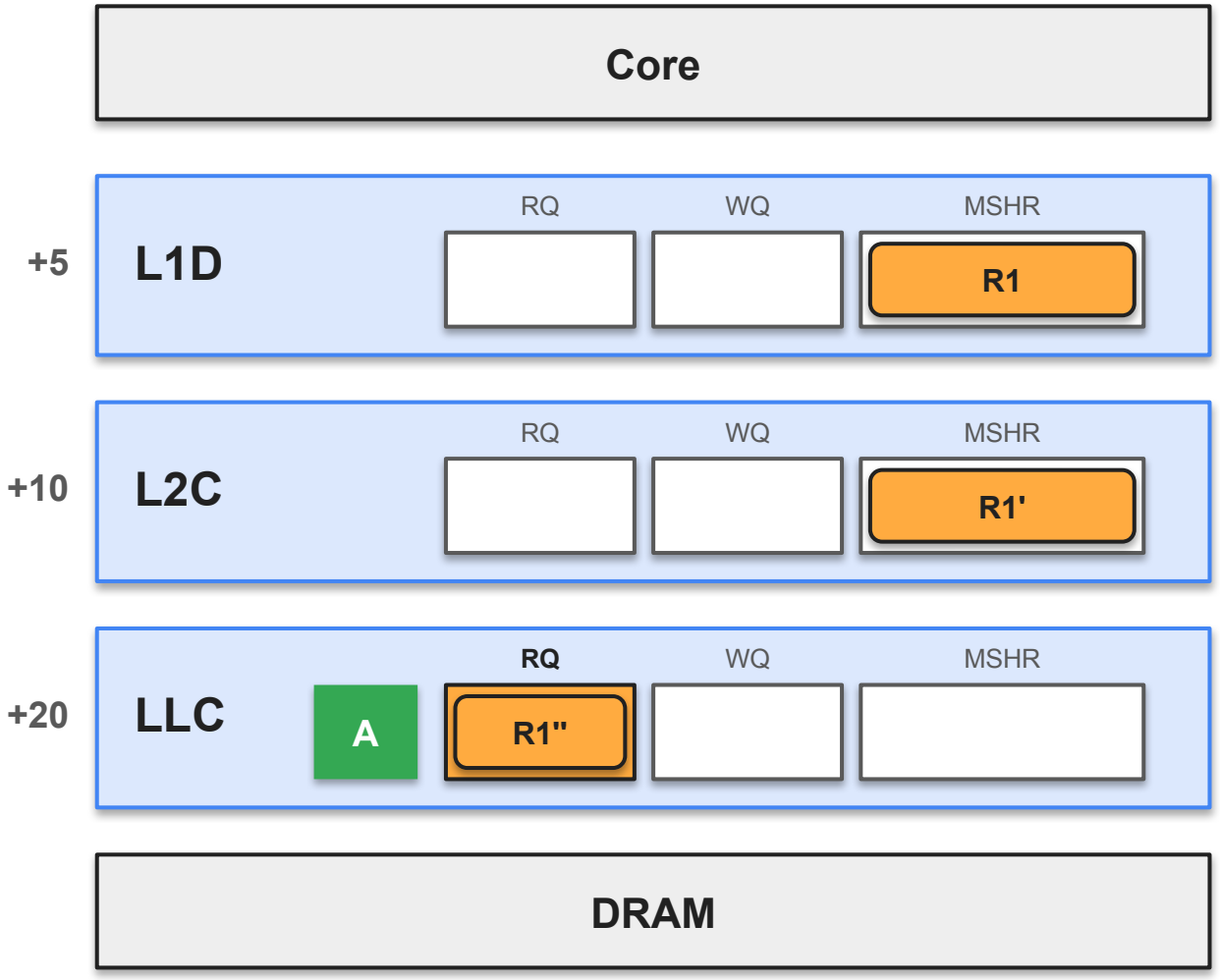




1024 sets in the L2C

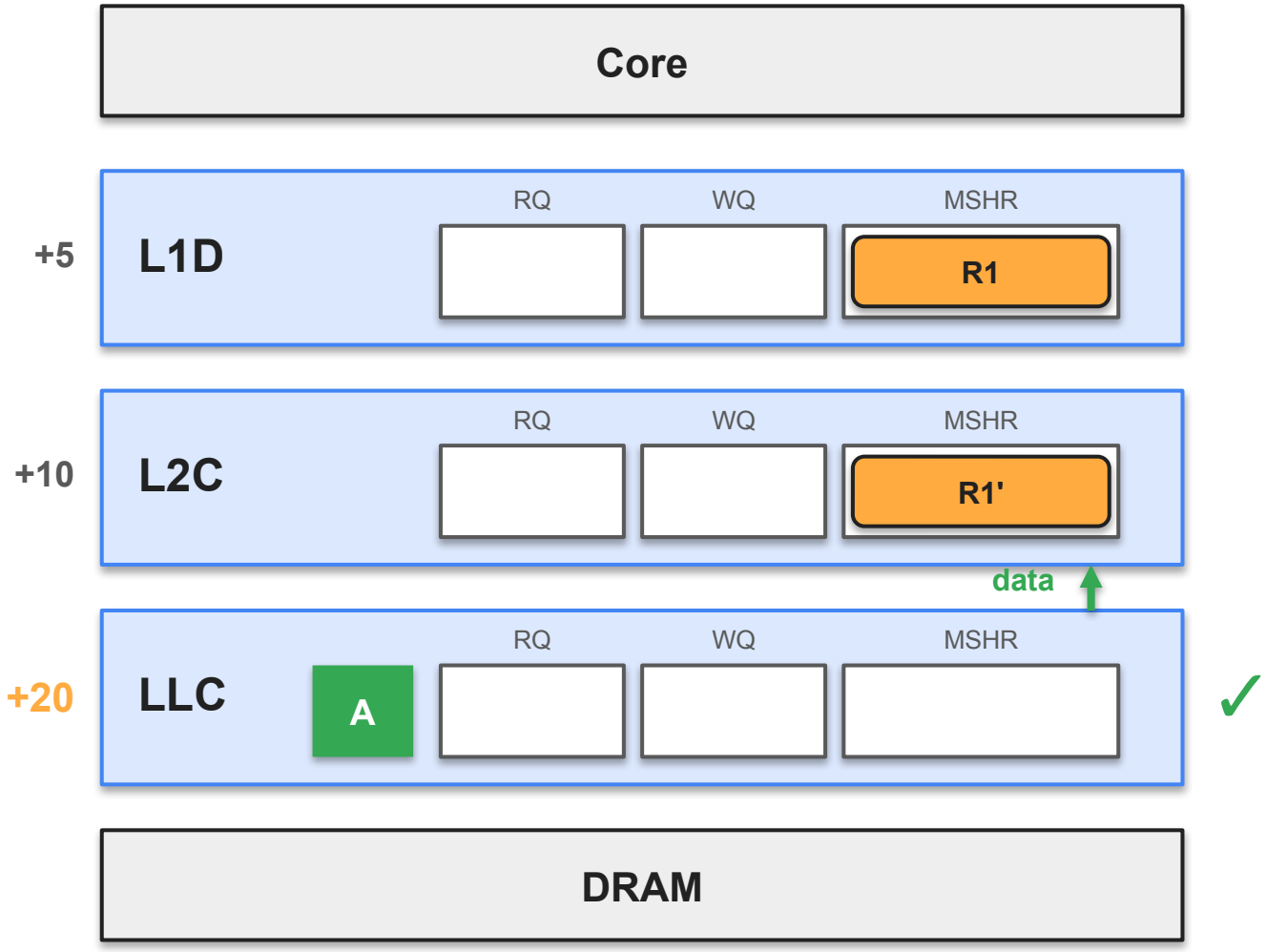
- The L2C does the same lookup in its own sets.
- No tag matches here either: **miss**!
- R1' moves to an L2C MSHR, and the L2C sends **R1"** down to the LLC.

R1" enters the LLC



- R1" waits in the LLC read queue.

LLC lookup: hit

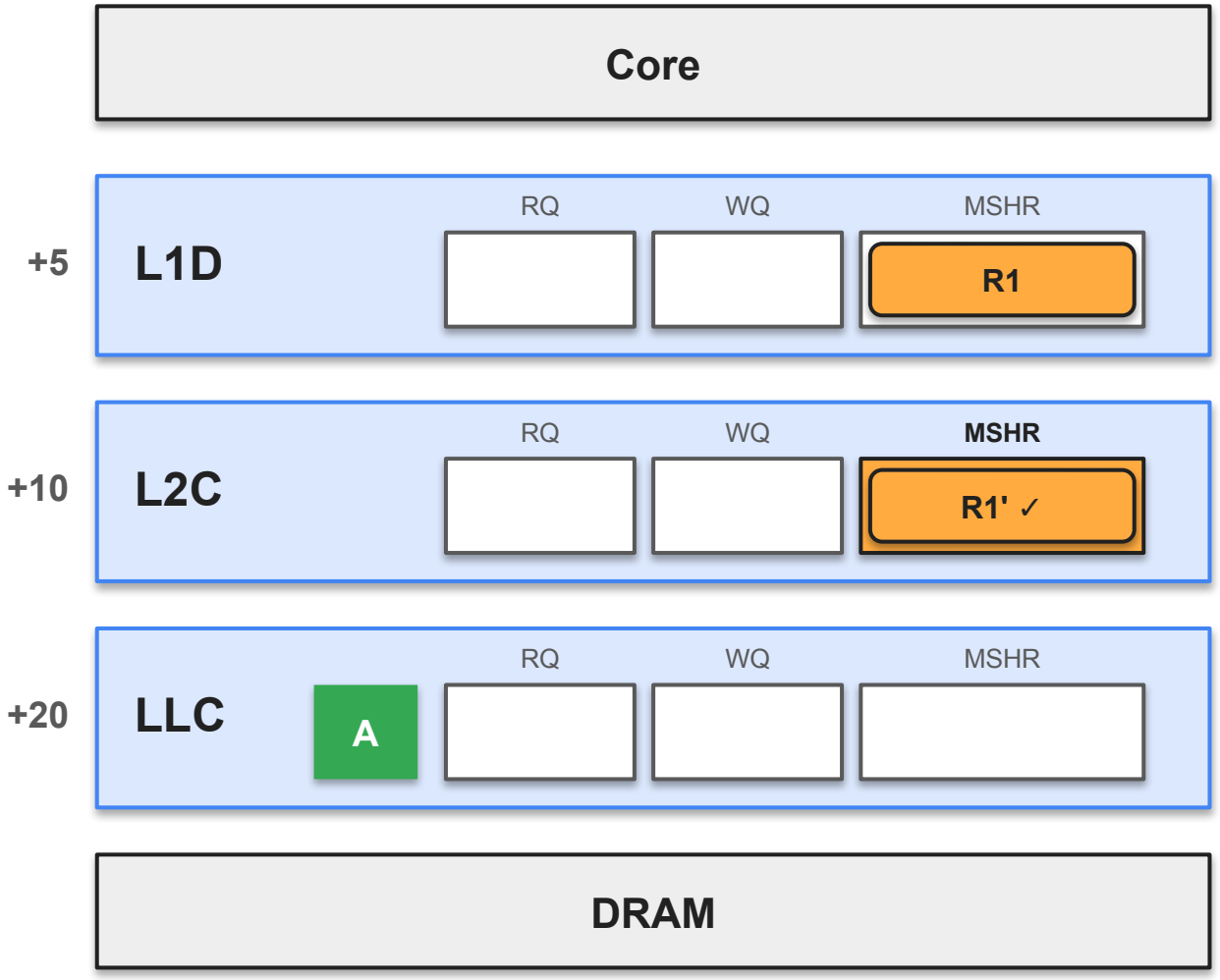




2048 sets in the LLC

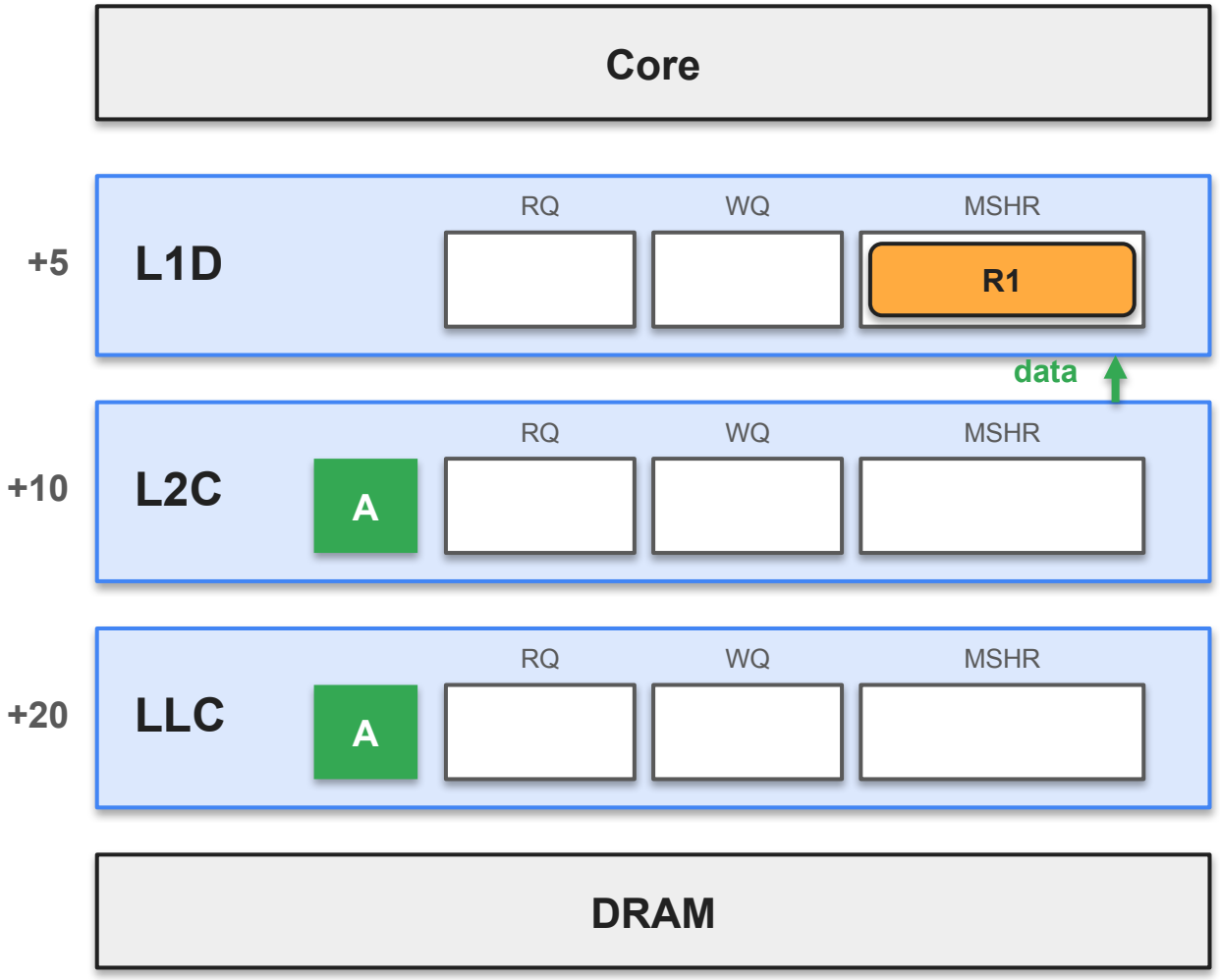
- The LLC finds a matching tag in the set: **hit**!
- The LLC sends the data up to the L2C.

The data reaches the L2C



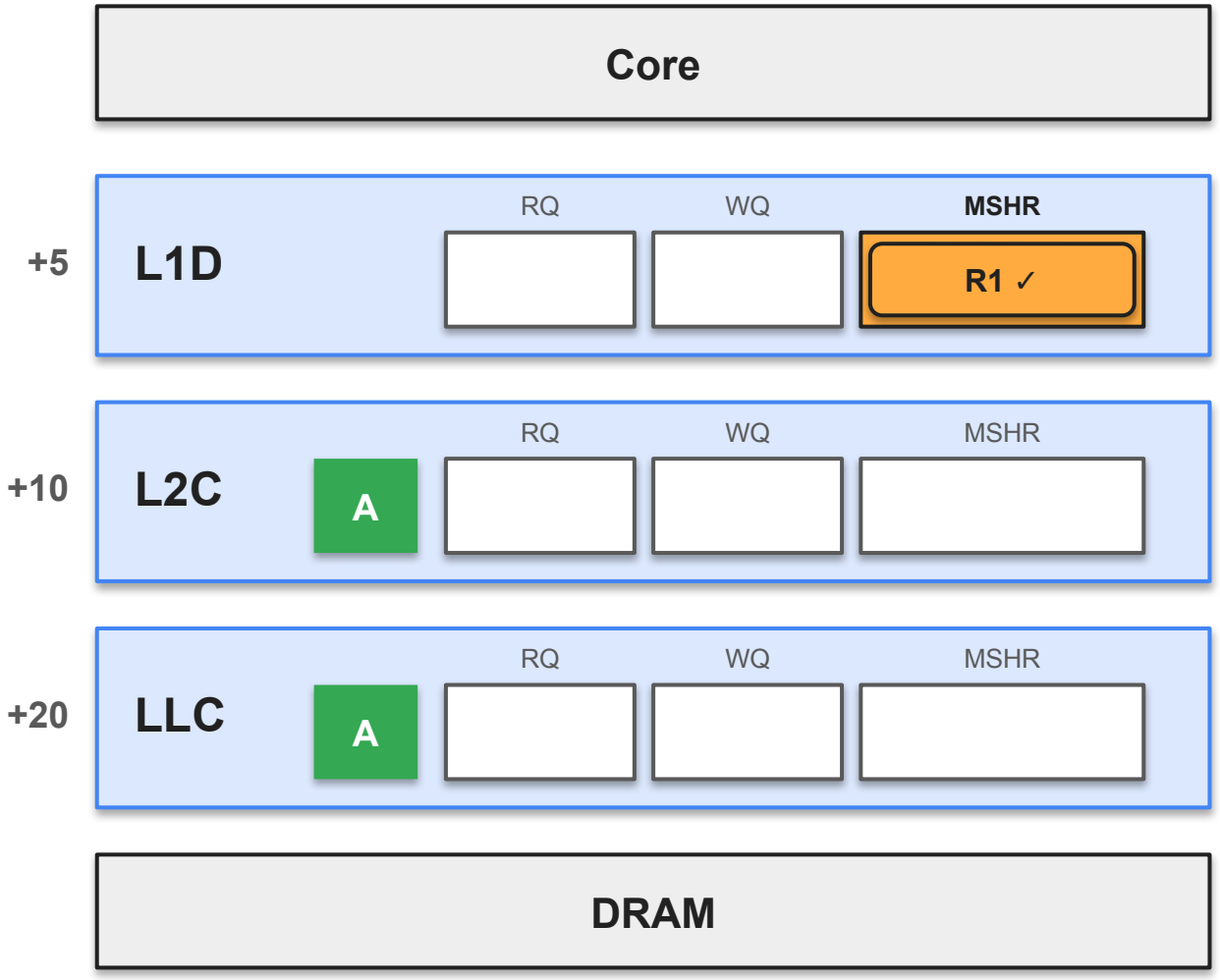
- The response finds the MSHR entry that waits for line A.
- That entry is now ready to fill.

The L2C fills the line



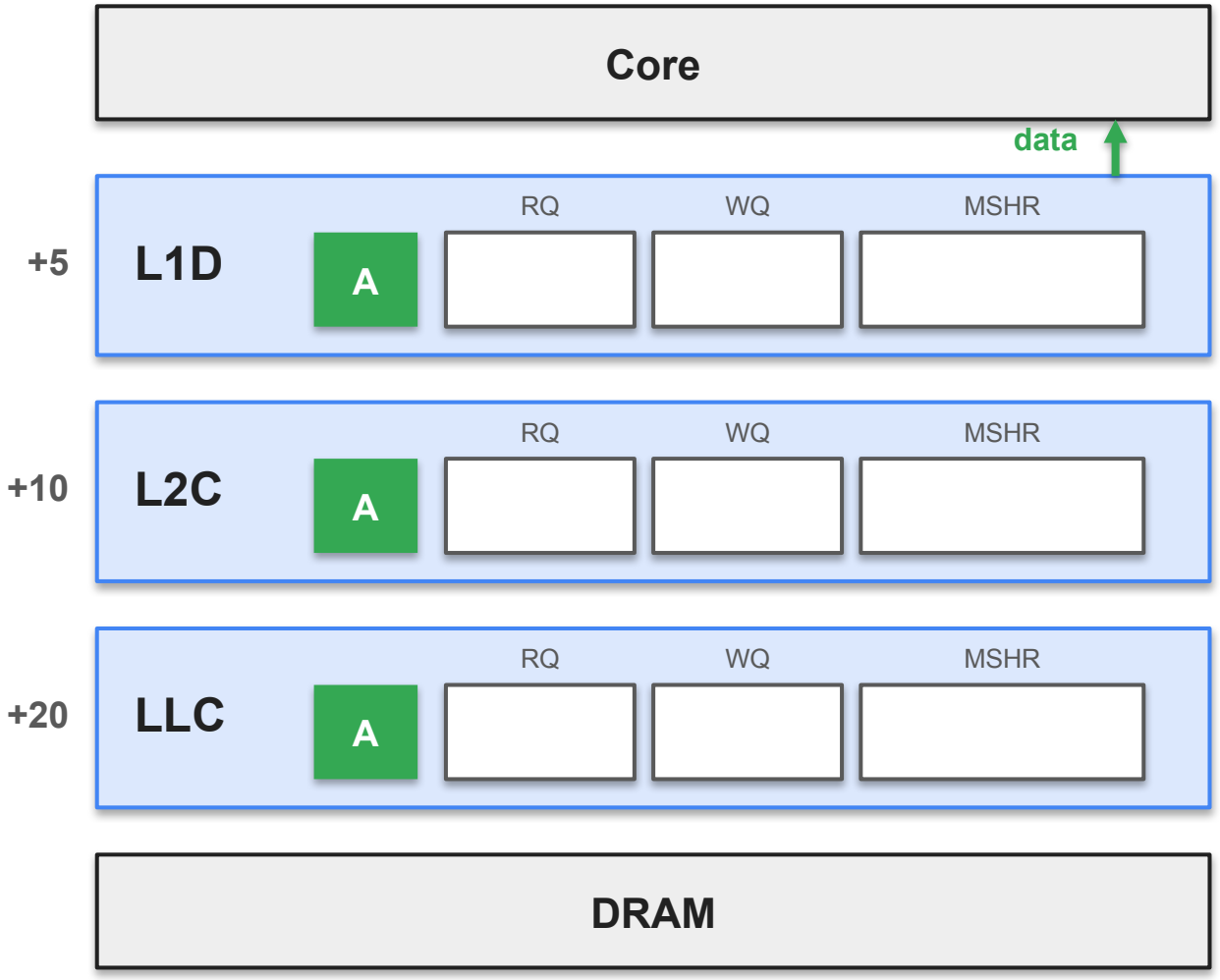
- The L2C writes line A into the set.
- It frees the MSHR entry and sends the data up to the L1D.

The data reaches the L1D



- The L1D MSHR entry for line A is now ready.

The L1D fills the line



- The L1D writes line A and frees its MSHR entry.
- The core gets its data, and R1 is done.
- The lookups alone take at least **5 + 10 + 20 = 35** cycles.

Packets

- Caches talk to each other with messages called **packets**.
- A cache sends a **request** down when it misses.
- The level below sends a **response** back up with the data.
- In ChampSim the response is the same packet, with the data filled in.

What a request carries

- **`address`**: the line we want. It gives the set and the tag.
- **`type`**: the kind of request (next slide).
- `ip`: the address of the instruction that made the access.
- `cpu`, `instr_id`: who waits for the data.
- `to_return`: the caches to answer.

What a response carries

- `address` and `type`: the same as in the request.
- `data`: the line itself.
- The cache sends the response to every cache listed in `to_return`.

Types of requests

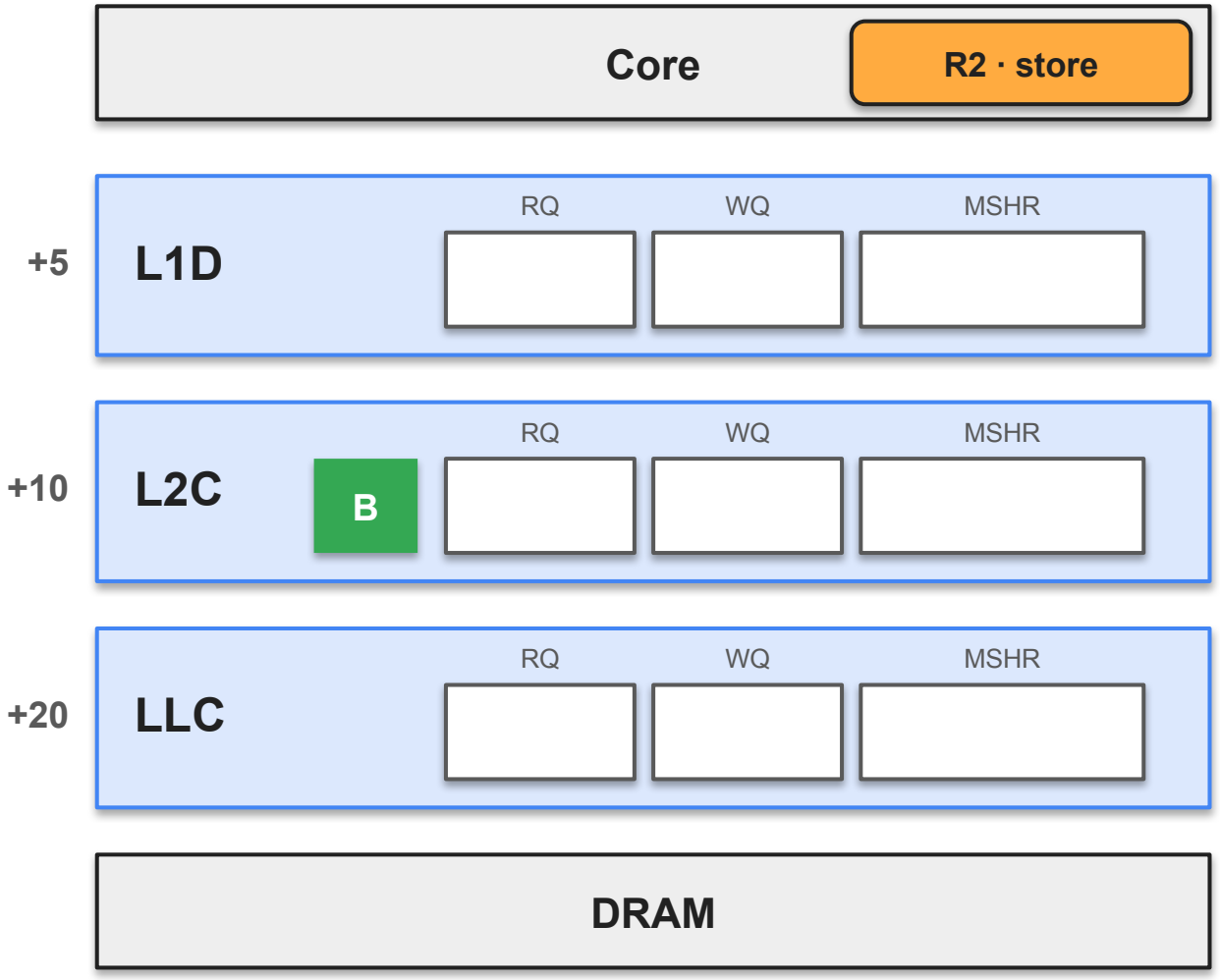
- `LOAD`: the program reads data. A load that the program asks for is a **demand load**.
- `RFO` (read for ownership): the program writes data. The cache must read the line first.
- `WRITEBACK`: the cache sends a modified line down when it evicts it.
- `PREFETCH`: a guess at what the program will need soon. It is off in this lab.
- `TRANSLATION`: page-table reads. You will see them in the OS course.

Before we trace a store

Q: I want to store to an address whose line is not in the L1D.

Q: What will happen?

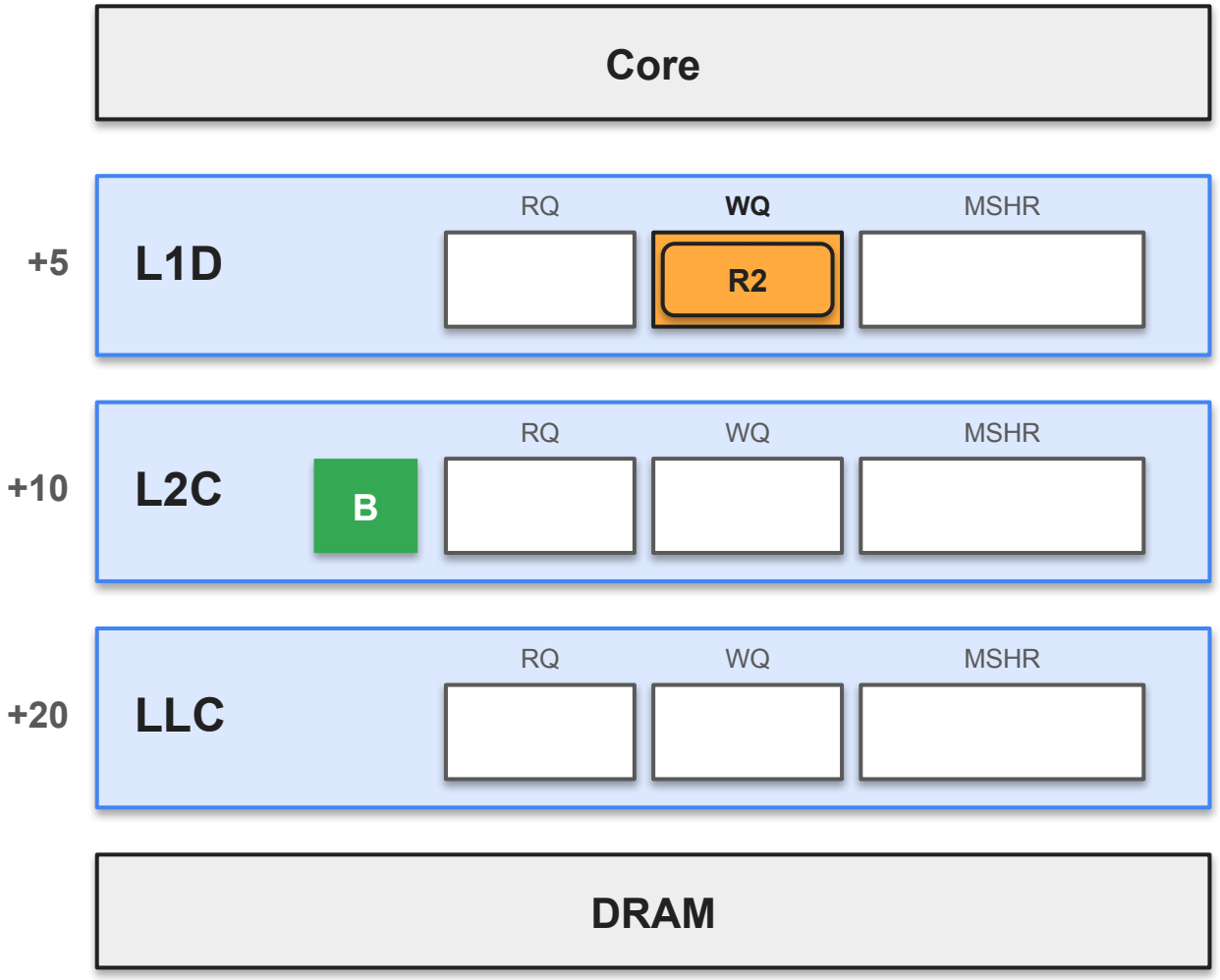
Tracing a store



- We follow one store, **R2**, to line B.
- Line B is in the L2C, but not in the L1D.
- Each packet now shows its type: LOAD, RFO or WRITEBACK.

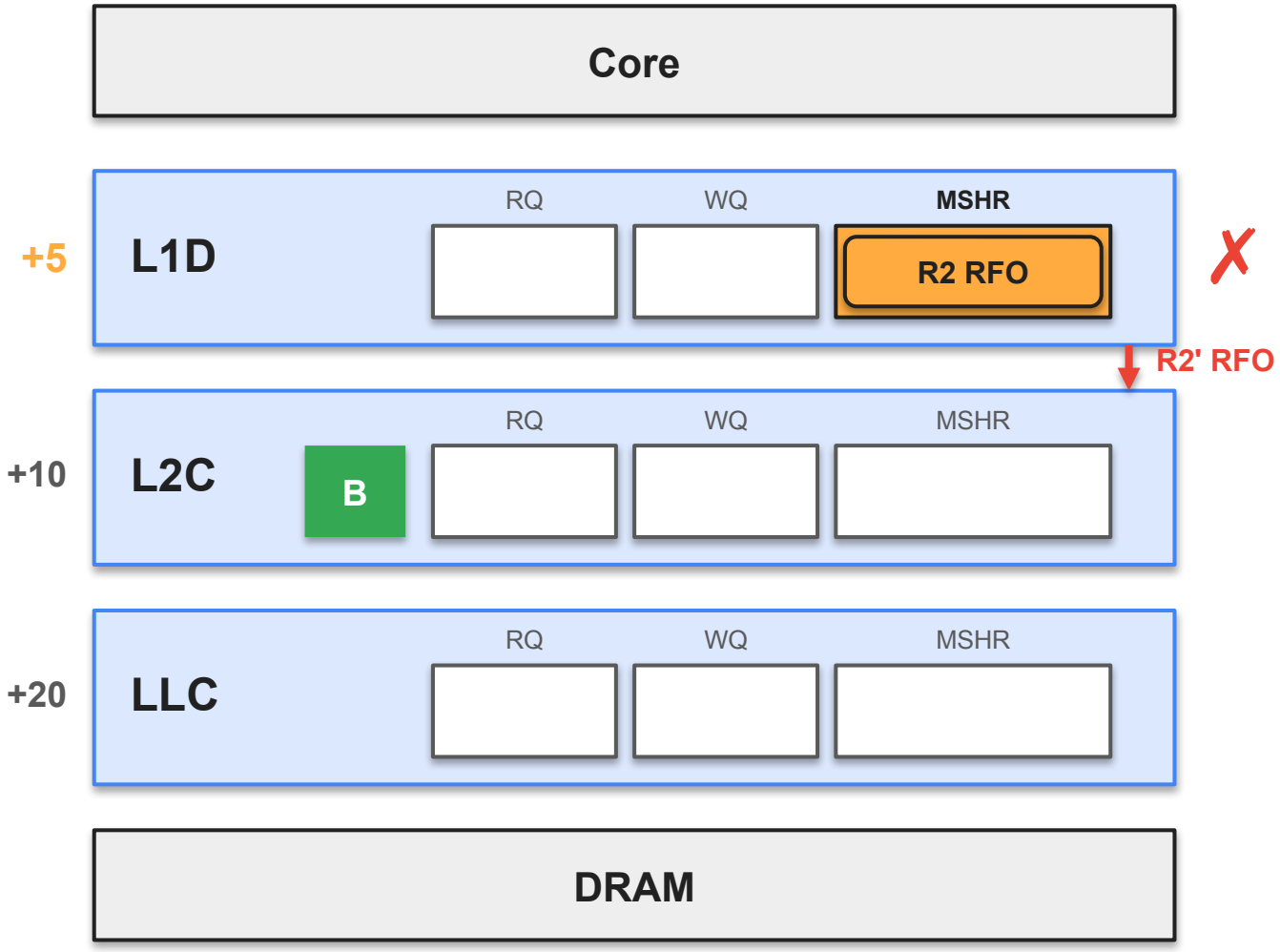
Q: What does it mean for a line to "reside" in the L2C?

R2 enters the L1D



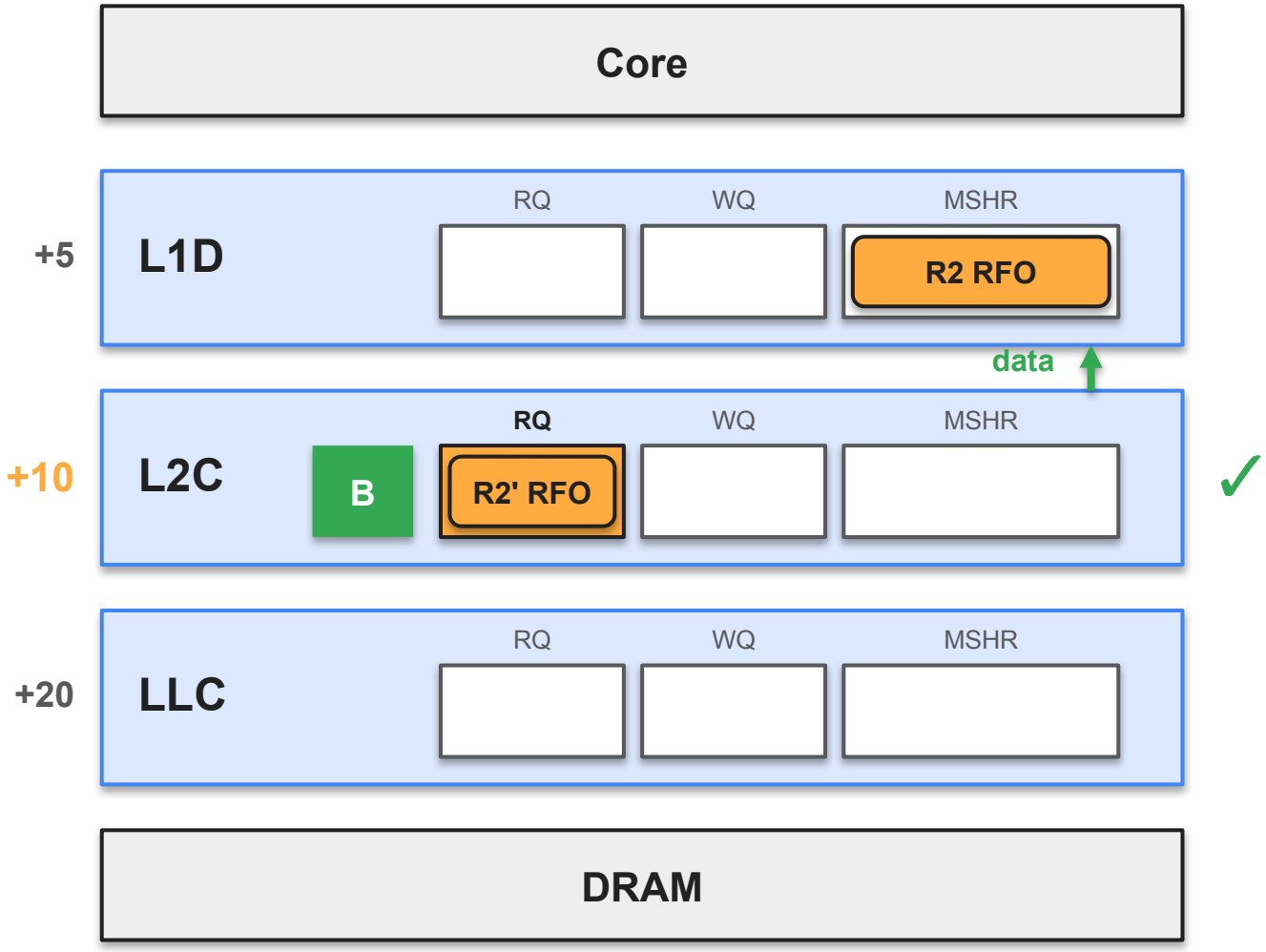
- Stores enter the L1D through its write queue (WQ).

L1D lookup: miss



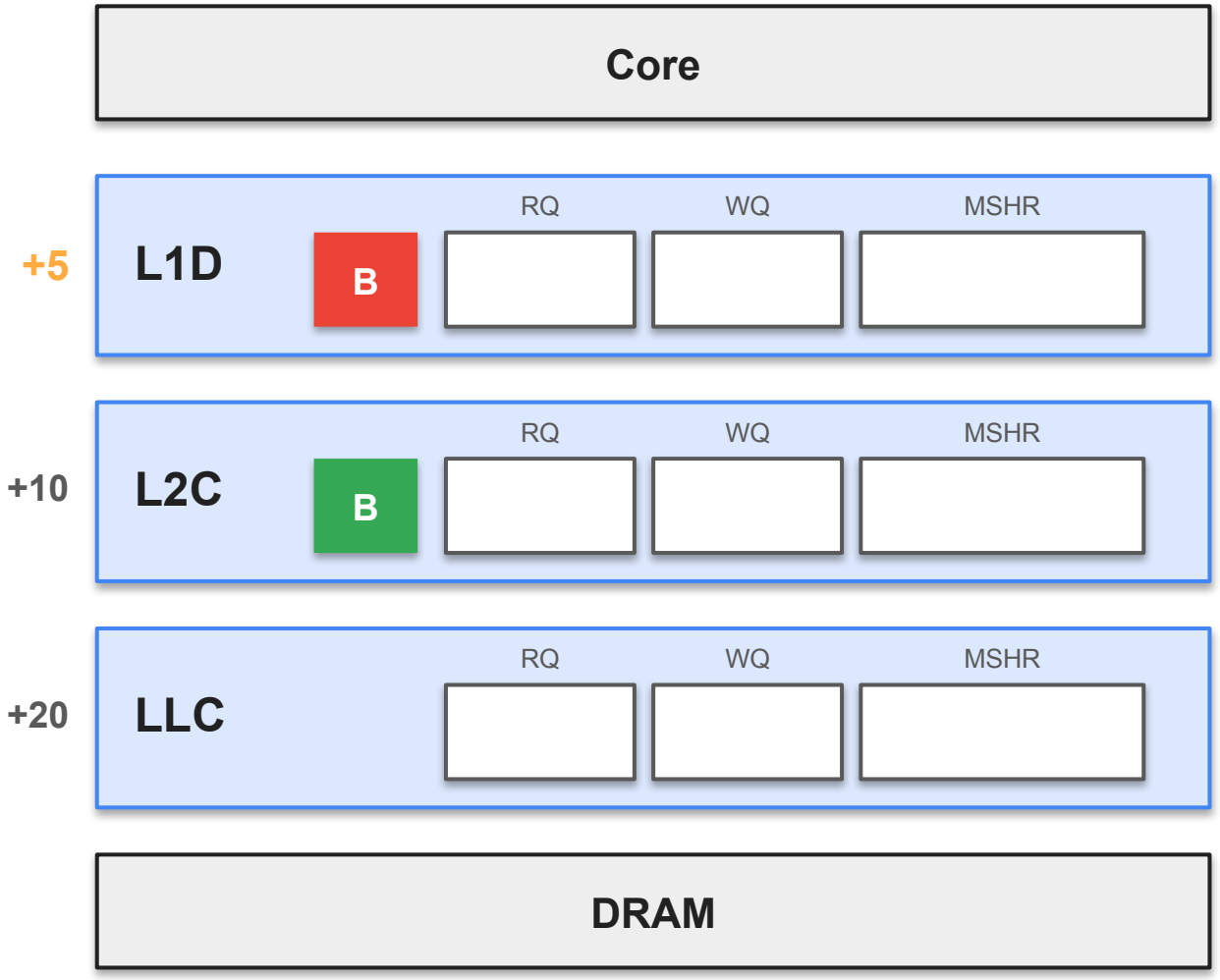
- Line B is not in the L1D: **miss**!
- The L1D must read line B before it can change it.
- R2 takes an MSHR, and the L1D sends **R2' (RFO)** down.

L2C lookup: hit



- R2' arrives in the L2C read queue.
- The L2C has line B: **hit**! It sends B up.

The L1D fills B and marks it dirty



- The L1D writes line B and applies the store.
- Line B now differs from the copy in the L2C, so we call it **dirty** (red).

Dirty lines

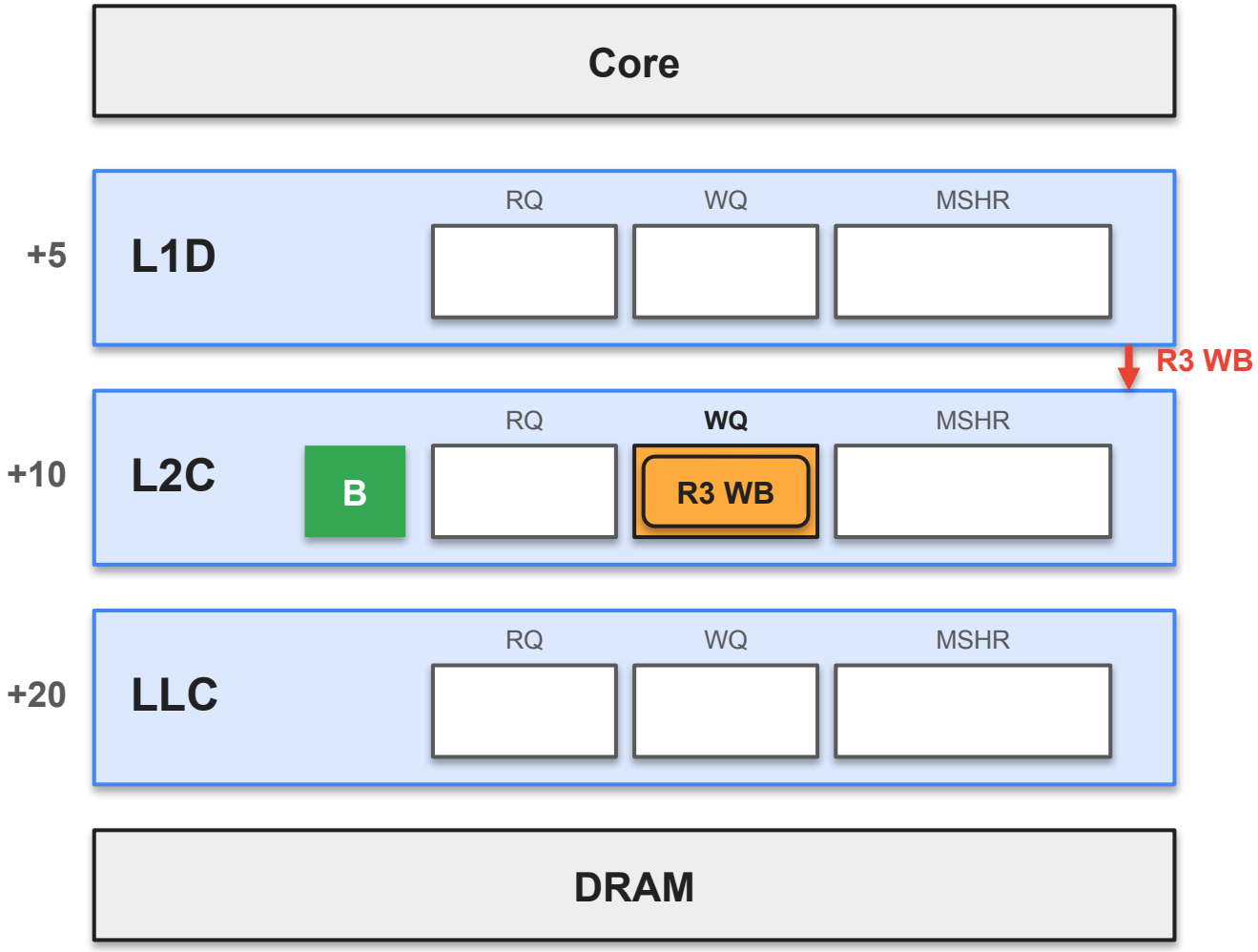
- A dirty line holds the only up-to-date copy of its data.
- When the cache evicts it, it must send it to the level below.

Q: If a clean line has to leave the cache, do I even need to send it to the level below?



* In an exclusive hierarchy the level below keeps no copy, so clean lines are sent down too.

Evicting B: a writeback



- Later, the L1D evicts line B.
- **R3 (WRITEBACK)** carries line B down to the L2C.
- R3 enters the L2C write queue, which holds writebacks.

Queues

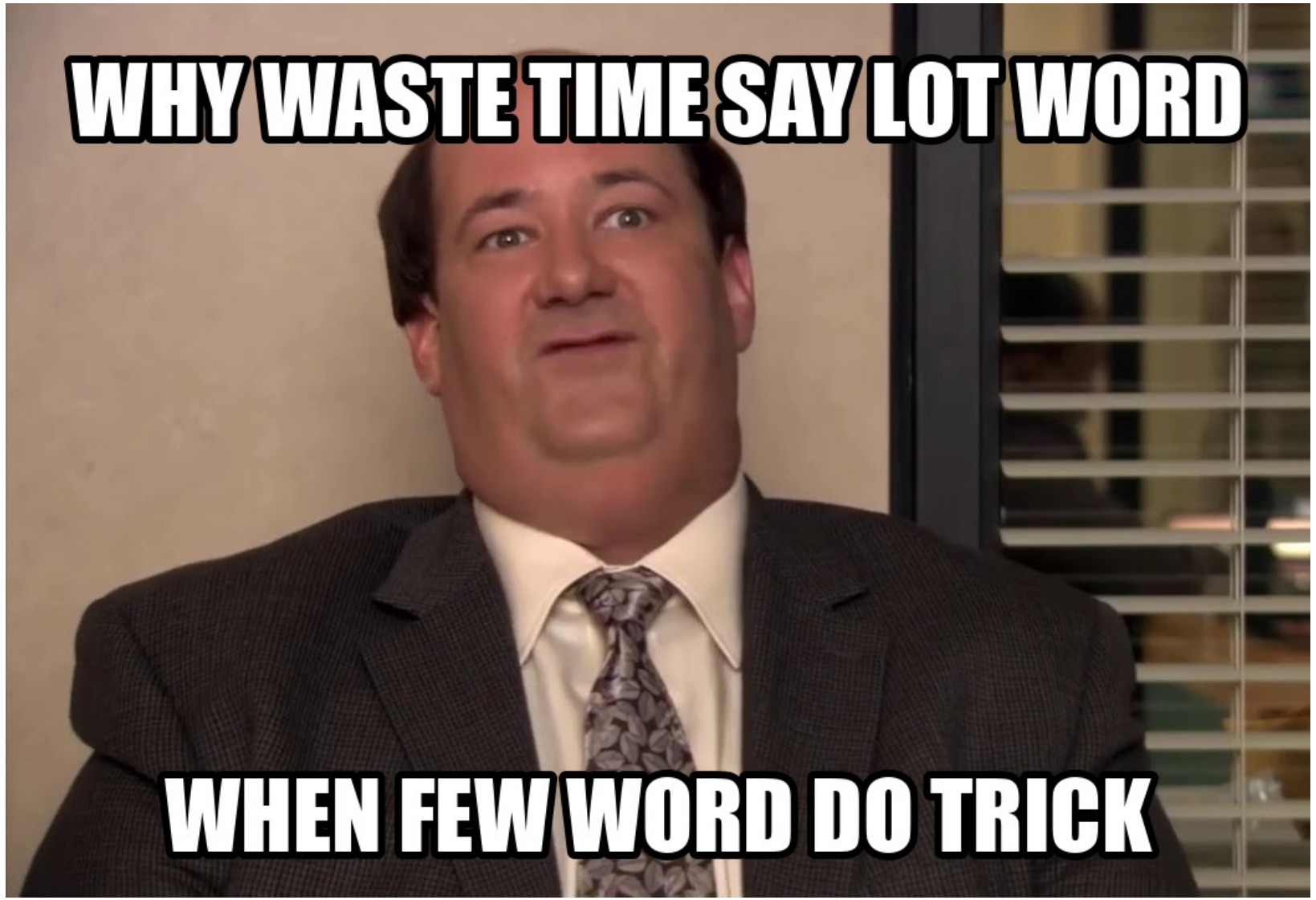
- **RQ** (read queue): loads and RFOs wait here to be served.
- **WQ** (write queue): at the L1D, stores from the core wait here.
- **Writeback queue**: dirty lines from the level above wait here. ChampSim uses the WQ for this too.

Q: Which cache level does not need a writeback queue?

MSHR

- A miss can take **hundreds** of cycles to return.
- If it waited in the RQ, every request behind it would wait too.
- So the cache moves it to an **MSHR** (miss status holding register) and serves the next request.
- A second miss to the same line joins the existing MSHR entry. (* Think about the massive benefits of this.)
- Any other miss gets a new entry.

Q: What happens if all MSHRs are full?


WHY WASTE TIME SAY LOT WORD
WHEN FEW WORD DO TRICK

Metrics

- **Miss rate** = misses / accesses.
  - The fraction of accesses that this cache could not serve.
- **MPKI** = misses per 1000 instructions.
  - Misses measured against the work the program does.
- **Average miss latency** = cycles from a miss until its data arrives.
  - ChampSim counts from when the request entered the queue until the fill, at that level only.
- LLC, `soplex`, 1M + 10M:
  - **286 951** misses of **504 999** accesses: miss rate **56.8%**, MPKI **28.7**, average miss latency **426** cycles

AMAT = hit time
+ miss rate × miss penalty

Average memory access time

Which metric?

- The miss rate depends on how many accesses reach the cache.
- MPKI compares the misses with the work the program does.
- The miss latency tells you how much each miss costs.

Q: I optimize a program P. Its miss rate increases but its MPKI decreases. Does that make sense?

Metrics at every level

```
CPU 0 cumulative IPC: 0.317071 instructions: 10000000 cycles: 31538718
cpu0_L1D TOTAL     ACCESS:    1489120  HIT:    1048653  MISS:     440467
cpu0_L1D AVERAGE MISS LATENCY: 305.874 cycles
cpu0_L2C TOTAL     ACCESS:     630218  HIT:     274097  MISS:     356121
cpu0_L2C AVERAGE MISS LATENCY: 364.649 cycles
LLC TOTAL     ACCESS:     504999  HIT:     218048  MISS:     286951
LLC AVERAGE MISS LATENCY: 426.488 cycles
```

- We can compute the same metrics at every cache level.
- Each level only sees the misses of the level above it.

Q: Why are the L2C and LLC miss rates so much higher than the L1D miss rate?

Metrics at every level: computed

← back to Metrics

Level	Accesses	Misses	Miss rate	MPKI	Avg miss latency
L1D	1 489 120	440 467	**29.6%**	**44.0**	306 cycles
L2C	630 218	356 121	**56.5%**	**35.6**	365 cycles
LLC	504 999	286 951	**56.8%**	**28.7**	426 cycles

- As seen by the core, **286 951** of **1 489 120** accesses go all the way to DRAM: **19.3%**.
- Fewer misses usually mean fewer cycles.

Q: Fewer misses: did the programmer or compiler change the program, or did the hardware improve? In the first case IPC can go either way. In the second case IPC goes up.

Branch statistics

```
CPU 0 Branch Prediction Accuracy: 92.9366% MPKI: 16.5733 Average ROB Occupancy at Mispredict: 31.4444
Branch type MPKI
BRANCH_DIRECT_JUMP: 0.0051
BRANCH_INDIRECT: 0.0004
BRANCH_CONDITIONAL: 16.5589
BRANCH_DIRECT_CALL: 0.003
BRANCH_INDIRECT_CALL: 0.0013
BRANCH_RETURN: 0.0046
```

- **Accuracy**: the fraction of branches whose direction and target were predicted correctly.
- **MPKI**: mispredictions per 1000 instructions. The rows below split it by branch type; they add up to it.
- **Conditional** branches (`if`, loop exits) cause almost all mispredictions. Direct jumps and calls have their target in the instruction; indirect ones read it from a register or memory; a return goes back to its caller.
- Ignore the ROB occupancy number.

Part 3

Replacement policies

- Sets and ways
- Choosing a victim
- LRU

Recall: set-associative caches

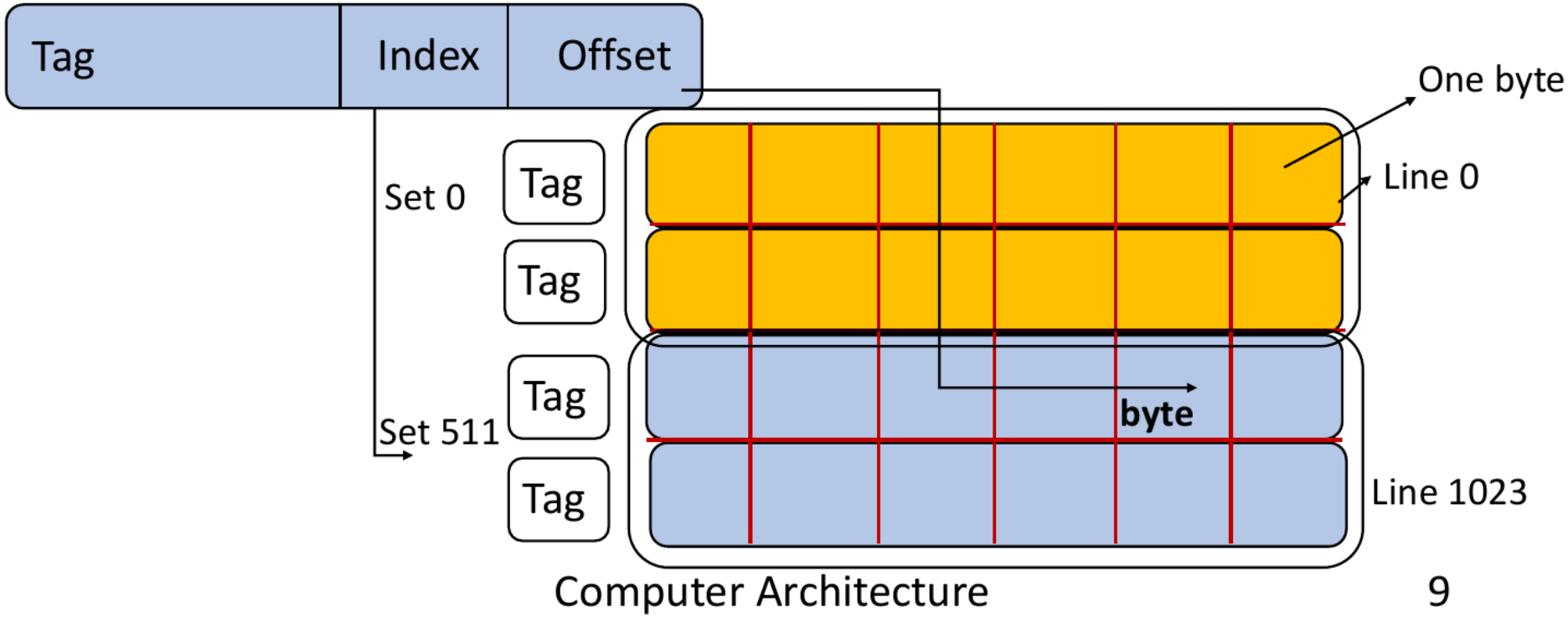
- We covered sets, ways and the bits of an address at the start of Part 2.
- The next two slides repeat the key pictures.

→ Recap: sets and ways

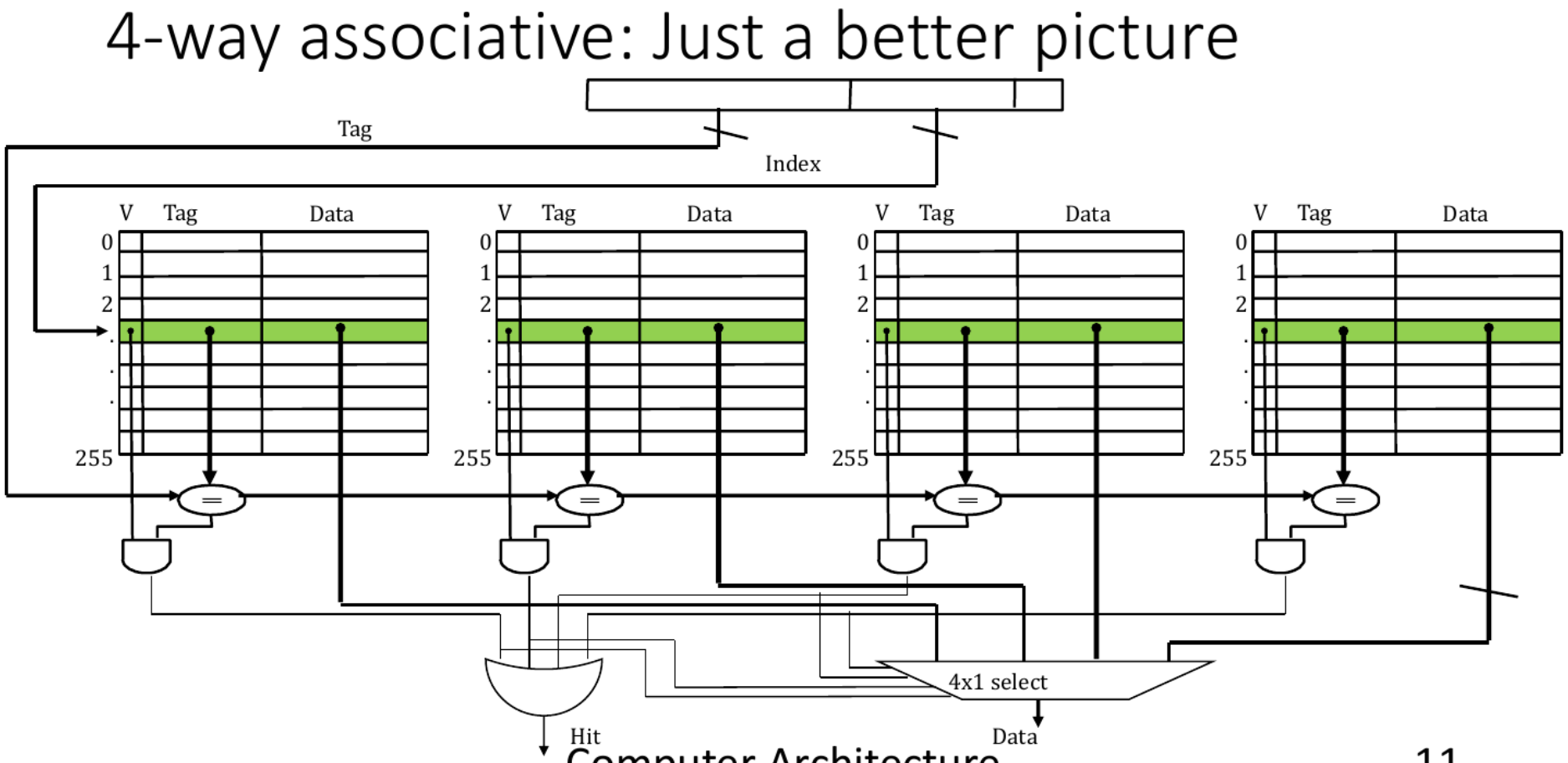
→ Recap: the bits of an address

Recall: sets and ways

What if we have multiple ways?



Recall: a 4-way set-associative cache



Which line do we replace?

On a Miss, Replace a block, which block?

Think of each block in a set having a “priority”
Indicating how important it is to keep the block in the cache

Key issue: How do you determine/adjust block priorities?

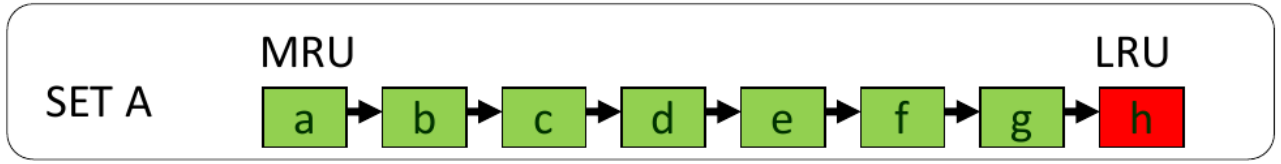
Ideally: Belady’s OPT policy, replace the block that will be used furthest in the future. No one knows the future though ☺

There are three key decisions in a set:
Insertion, promotion, eviction (replacement)

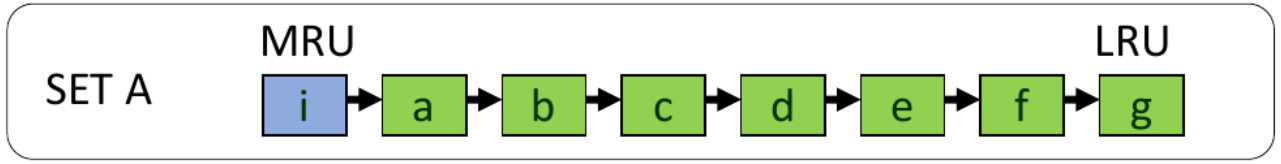
LRU

A simple LRU (Least-Recently-Used) Policy

Cache Eviction Policy: On a miss (block *i*), which block to evict (replace) ?



Cache Insertion Policy: New block *i* inserted into MRU.



Cache Promotion Policy: On a future hit (block *i*), promote to MRU

We need priority bits per block. For example, a 16-way cache will need four bit/block LRU causes thrashing when working set > cache size

Replacement policies

- A set has a fixed number of ways. When a new line arrives and the set is full, one line must leave.
- A replacement policy decides three things:
  - **Insertion**: where a new line starts.
  - **Promotion**: what a hit does to a line.
  - **Eviction**: which line leaves.
- Programs tend to reuse the data they used recently. LRU bets on this: the line that has not been used for the longest time is the least likely to be needed soon.
- So LRU inserts and promotes a line as the most recent, and evicts the least recent.
- An L1 has only a few cycles per access, so its policy must be cheap. Lower levels can afford more state.
- There are many replacement policies. We discuss a few of them. A policy can also choose to keep no copy at all (bypassing, see the extra slides).

Review: you can now answer

- What are request and response packets, and what information do they carry?
- Why do caches need an RQ, a WQ and MSHRs?
- What are LOAD, RFO and WRITEBACK requests?
- Why does a full set need a replacement policy?
- What does LRU insert, promote and evict?
- What do miss rate, MPKI and miss latency measure?

Part 4

ChampSim under the hood

- The main loop and the output
- cache.cc: the read path
- Task 1
- Replacement code
- Tasks 2 and 3
- Reference: the write path

How ChampSim runs

```
// main.cc:414 (trimmed)
while (simulation not complete) {
    for (auto op : operables)        // cores, caches, DRAM
        op→_operate();               // one cycle each

    // read the next instructions from the trace
    ooo_cpu[i]→init_instruction(traces[i]→get());

    if (num_retired > warmup_instructions)
        finish_warmup();             // reset the statistics
}
// then print_roi_stats() for every cache
```

- Open `09_inlab/Champsim` and follow along.
- ChampSim builds the caches and cores from the JSON.
- In every cycle, each part runs one step.
- The warm-up runs first, then the counted instructions.

The output

```
$ ./bin/champsim --warmup_instructions 1000000 --simulation_instructions 10000000 /opt/CS231_Traces/soplex_66B.trace.xz
Warmup complete CPU 0 instructions: 1000002 cycles: 267780
Region of Interest Statistics
CPU 0 cumulative IPC: 0.317071 instructions: 10000000 cycles: 31538718
cpu0_L1D TOTAL     ACCESS:    1489120  HIT:    1048653  MISS:     440467
cpu0_L1D LOAD      ACCESS:     977101  HIT:     596896  MISS:     380205
cpu0_L1D RFO       ACCESS:     502094  HIT:     442134  MISS:      59960
cpu0_L2C TOTAL     ACCESS:     630218  HIT:     274097  MISS:     356121
cpu0_L2C WRITEBACK ACCESS:     189522  HIT:     189474  MISS:         48
LLC TOTAL     ACCESS:     504999  HIT:     218048  MISS:     286951
LLC LOAD      ACCESS:     305667  HIT:      57893  MISS:     247774
LLC RFO       ACCESS:      50325  HIT:      11224  MISS:      39101
LLC WRITEBACK ACCESS:     148931  HIT:     148895  MISS:         36
LLC AVERAGE MISS LATENCY: 426.488 cycles
CPU 0 Branch Prediction Accuracy: 92.9366% MPKI: 16.5733
BRANCH_CONDITIONAL: 16.5589
```

H: We will demystify what these terms and outputs mean. IPC = instructions per cycle: higher is better.

What the output rows count

```
cpu0_L1D TOTAL      ACCESS:    1489120  HIT:    1048653  MISS:     440467
cpu0_L1D LOAD       ACCESS:     977101  HIT:     596896  MISS:     380205
cpu0_L1D RFO        ACCESS:     502094  HIT:     442134  MISS:      59960
cpu0_L1D WRITEBACK  ACCESS:          0  HIT:          0  MISS:          0
cpu0_L1D TRANSLATION ACCESS:       9925  HIT:       9623  MISS:        302
cpu0_L2C LOAD       ACCESS:     380439  HIT:      74768  MISS:     305671
cpu0_L2C WRITEBACK  ACCESS:     189522  HIT:     189474  MISS:         48
```

- **TOTAL** adds up the rows below it.
- **LOAD**: at the L1D, the program's loads. At the L2C, the load misses of the L1D **and** of the L1I, so the rows do not map one to one.
- **RFO**: at the L1D, every store, hit or miss. Below it, the RFO misses of the level above.
- **WRITEBACK**: dirty lines from the level above. Always **0** at the L1D.
- `TRANSLATION`: page-table reads, which also pass through the L1D. `PREFETCH` rows stay **0**: prefetching is off.

The CACHE class (inc/cache.h)

```
class CACHE : public champsim::operable, ... {
public:
  const std::string NAME;                          // "cpu0_L1D", "cpu0_L2C", "LLC"
  const uint32_t NUM_SET, NUM_WAY, ...;            // the shape, from the JSON
  // line (set, way) is block[set * NUM_WAY + way]
  std::vector<BLOCK> block{NUM_SET * NUM_WAY};

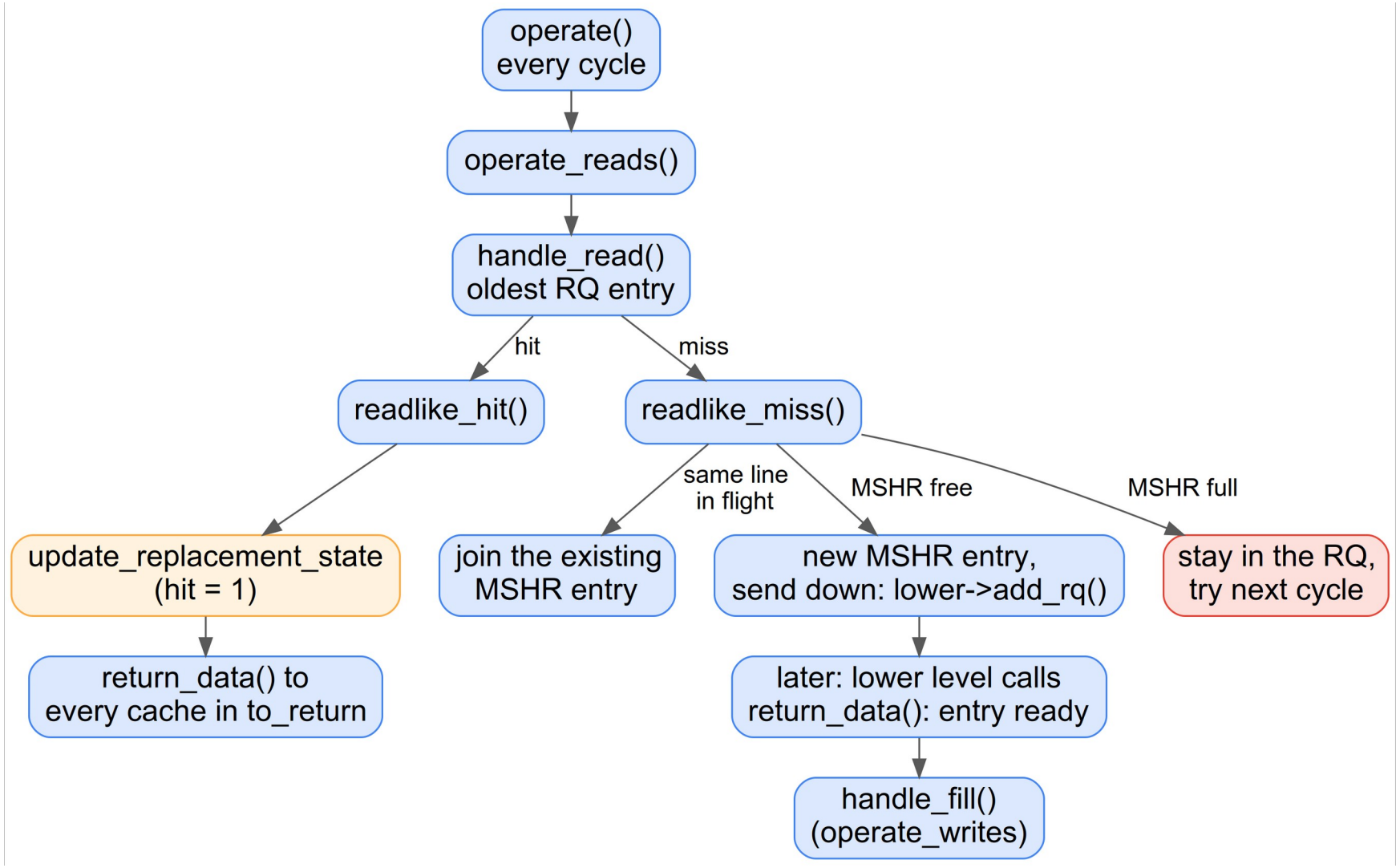
  champsim::delay_queue<PACKET> RQ, PQ, VAPQ, WQ;   // read, prefetch, write queues
  std::list<PACKET> MSHR;                          // outstanding misses

  uint64_t sim_access[..][..], sim_hit[..][..], sim_miss[..][..];

  void operate() override;                         // one cycle
  void handle_fill(); void handle_writeback(); void handle_read(); ...
};
```

H: Every cache in the hierarchy is one CACHE object. Everything in cache.cc works on these members.

The read path in cache.cc



cache.cc: operate

- In every cycle, ChampSim calls `operate()` on each cache. It first handles fills and writebacks, then reads.

```
void CACHE::operate() {
    operate_writes();   // handle_fill, handle_writeback
    operate_reads();    // handle_read, handle_prefetch
}
```

cache.cc: handle_read

- `handle_read` takes the oldest request from the RQ. On a hit it calls `readlike_hit`; on a miss it calls `readlike_miss`.

```
void CACHE::handle_read()                    // cache.cc:122
```

cache.cc: readlike_miss

- `readlike_miss` joins the miss to an MSHR entry for the same line. If there is none, it allocates one and sends a request down.

```
bool CACHE::readlike_miss(PACKET& handle_pkt)    // cache.cc:220
```

cache.cc: return_data

- The lower level calls `return_data` when the data comes back. It marks the matching MSHR entry as ready.

```
void CACHE::return_data(PACKET* packet)    // cache.cc:655
```

cache.cc: handle_fill

- `handle_fill` takes a ready MSHR entry, writes the line into the set and answers the level above.

```
void CACHE::handle_fill()                    // cache.cc:34
```

handle_read

```
void CACHE::handle_read() {
  while (reads_available_this_cycle > 0) {    // bandwidth: max_read
    if (!RQ.has_ready()) return;
    PACKET& handle_pkt = RQ.front();          // oldest request

    uint32_t set = get_set(handle_pkt.address);
    uint32_t way = get_way(handle_pkt.address, set);  // NUM_WAY if absent

    if (way < NUM_WAY)                        // HIT
      readlike_hit(set, way, handle_pkt);
    else if (!readlike_miss(handle_pkt))      // MISS
      return;                                 // no MSHR free: try next cycle

    RQ.pop_front();
    reads_available_this_cycle--;
  }
}
```

- `get_way` returns `NUM_WAY` when the line is not in the set.
- A miss that cannot get an MSHR stays in the RQ.

Hit: readlike_hit

```
void CACHE::readlike_hit(set, way, PACKET& handle_pkt) {
  BLOCK& hit_block = block[set * NUM_WAY + way];
  handle_pkt.data = hit_block.data;

  // tell the replacement policy about the hit
  impl_replacement_update_state(..., set, way, ..., /*hit*/ 1);

  sim_hit[cpu][type]++;   sim_access[cpu][type]++;

  for (auto ret : handle_pkt.to_return)     // send the response up
    ret->return_data(&handle_pkt);
}
```

- We copy the data into the packet.
- `impl_replacement_update_state` tells the replacement policy that this line was hit.
- We count the hit.
- `return_data` hands the packet to every cache in `to_return`.

Miss: readlike_miss

```
bool CACHE::readlike_miss(PACKET& handle_pkt) {
  auto mshr_entry = find in MSHR (same address);
  if (mshr_entry != MSHR.end()) {          // miss already in flight
    merge handle_pkt.to_return into mshr_entry;
  } else {
    if (MSHR.size() == MSHR_SIZE) return false;   // MSHR full
    if (lower level RQ is full)    return false;

    // fill_level can skip the entry: then this level keeps no copy
    MSHR.insert(std::end(MSHR), handle_pkt);      // new entry
    handle_pkt.to_return = {this};                // answer me
    lower_level->add_rq(&handle_pkt);             // send it down
  }
  return true;
}
```

- If the same line is already missing, we join that entry.
- Otherwise we allocate an entry and send a request down.
- When we return `false`, the packet stays in the RQ.

Servicing the miss: handle_fill

```
void CACHE::handle_fill() {
  while (writes_available_this_cycle > 0) {    // bandwidth: max_write
    auto fill_mshr = MSHR.begin();
    if (fill_mshr not ready) return;           // data not back yet

    uint32_t set = get_set(fill_mshr->address);
    way = first invalid way in the set;
    if (way == NUM_WAY)                        // set is full
      way = impl_replacement_find_victim(...);

    if (!filllike_miss(set, way, *fill_mshr)) return;

    for (auto ret : fill_mshr->to_return)      // send the response up
      ret->return_data(&(*fill_mshr));
    MSHR.erase(fill_mshr);
    writes_available_this_cycle--;
  }
}
```

- We use an empty way first.
- Only a full set asks the policy for a victim.
- Then every waiting cache gets the data.

Filling the line: filllike_miss

```
bool CACHE::filllike_miss(set, way, PACKET& handle_pkt) {
  BLOCK& fill_block = block[set * NUM_WAY + way];

  if (fill_block.dirty) {                    // victim was modified
    PACKET writeback_packet;   ... type = WRITEBACK;
    if (lower_level->add_wq(&writeback_packet) == -2)
      return false;                          // lower WQ full
  }

  fill_block.valid = true;
  fill_block.dirty = (type == WRITEBACK || (type == RFO && to_return.empty()));
  fill_block.address = handle_pkt.address;    ... // copy the rest

  impl_replacement_update_state(..., set, way, ..., /*hit*/ 0);
  sim_miss[cpu][type]++;   sim_access[cpu][type]++;
  return true;
}
```

- A dirty victim goes down as a writeback first.
- The new line is dirty if it carries a store.
- We tell the policy about the fill.

It's inlab (1) time

Task 1: count events in the L2C

- Count the read hits in each way of the L2C.
- Optional: count the misses in each set and print the largest count.
- A read is a request that comes through the RQ: LOAD and RFO. TRANSLATION reads also count, but we do not teach them.
- Edit the copies in `task1/`: `cache.h`, `cache.cc` and `main.cc`. Then run `make task1` and `make check-task1`.

Q: Where is the cache's state declared?

Q: Where is a read hit detected? Where is a miss counted?

Q: Where are the statistics printed?

You are replaceable

- Each policy is a folder: `replacement/<name>/<name>.cc`.
- The file defines four functions of `CACHE`:
  - `initialize_replacement()`: once, at the start
  - `find_victim(...)`: returns the way to evict
  - `update_replacement_state(...)`: runs on every hit and every fill
  - `replacement_final_stats()`: once, at the end
- Set `"replacement": "<name>"` in the JSON and rebuild.

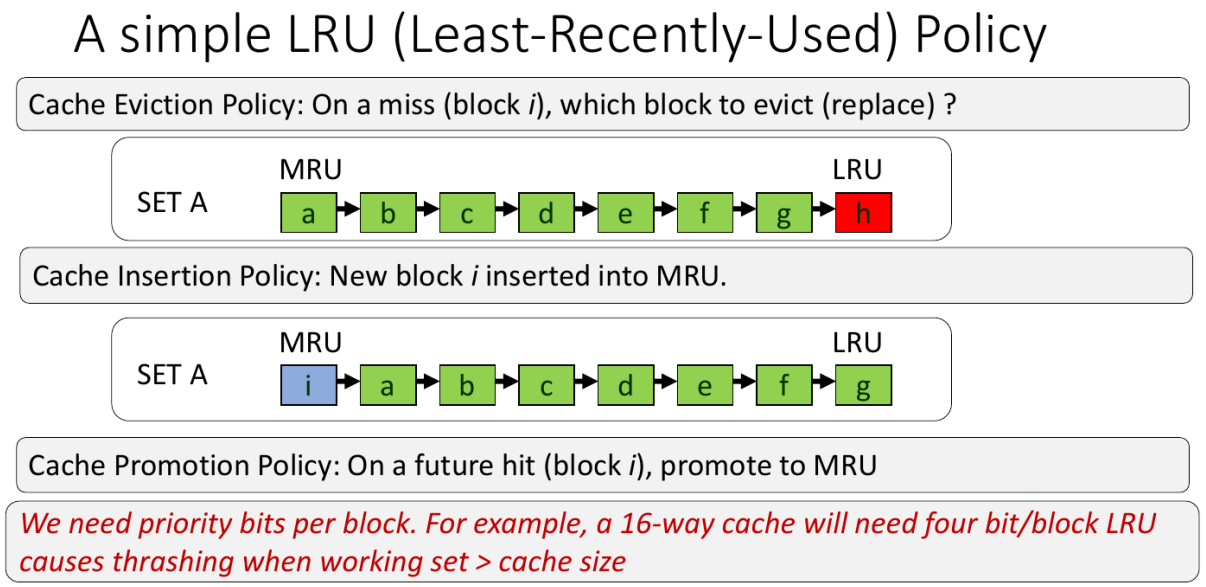
Where the policy is called

- `readlike_hit` → `update_replacement_state(hit = 1)`
- `handle_fill` → `find_victim`, only if the set is full
- `filllike_miss` → `update_replacement_state(hit = 0)`
- Stores and writebacks call them in the same way.
- Prefetchers also call them. We switch prefetching off in this lab.

It's inlab (2) time

Task 2: from an idea to code

- This task shows how to turn an existing idea into code.
- The idea: LRU keeps the lines of a set in order of last use.
- A hit or a fill moves a line to the front (most recent).
- Eviction takes the line at the back (least recent).



Idea → code: the order of a set

```cpp
std::map<CACHE*, std::vector<uint32_t>> lruBits;  // one age per line

void CACHE::initialize_replacement() {
  lruBits[this] = std::vector<uint32_t>(NUM_SET * NUM_WAY, 0);
}
```

- We store the order as an **age** for every line.
- Age `NUM_WAY - 1` is the most recent line; age `0` is the least recent.
- Each cache that uses the policy gets its own vector.

Idea → code: eviction

```
uint32_t CACHE::find_victim(..., uint32_t set, ...) {
  for (uint32_t i = 0; i < NUM_WAY; i++)
    if (lruBits[this][set * NUM_WAY + i] == 0)   // least recent
      return i;
}
```

- The idea says: evict the line at the back.
- In code, we return the way whose age is `0`.

Idea → code: insertion and promotion

```
void CACHE::update_replacement_state(..., set, way, ..., type, hit) {
  if (hit && type == WRITEBACK) return;          // not a real use

  uint32_t currVal = lruBits[this][set * NUM_WAY + way];
  for (uint32_t i = 0; i < NUM_WAY; i++)         // newer lines age by 1
    if (lruBits[this][set * NUM_WAY + i] > currVal)
      lruBits[this][set * NUM_WAY + i]--;

  lruBits[this][set * NUM_WAY + way] = NUM_WAY - 1;   // most recent
}
```

- The idea says: move the line to the front.
- In code, every line that was newer than it ages by one.
- Then the line gets the age `NUM_WAY - 1`.
- A fill (`hit = 0`) is the insertion, and a hit (`hit = 1`) is the promotion.

Debugging: follow one set

```
// at the end of update_replacement_state (needs #include <iostream>)
if (warmup_complete[cpu] && set == 5) {            // one set, after the warm-up
  std::cout << NAME << " set 5 " << (hit ? "hit " : "fill") << " way " << way
            << " line " << std::hex << (full_addr >> 6) << std::dec << " ages:";
  for (uint32_t i = 0; i < NUM_WAY; i++)
    std::cout << " " << lruBits[this][set * NUM_WAY + i];
  std::cout << std::endl;
}
```

- Print from inside the policy to watch it work. Print one set only, or the output floods.
- Run the binary yourself and filter: `Champsim/bin/mylru --warmup_instructions 1000000 --simulation_instructions 3000000 /opt/CS231_Traces/soplex_66B.trace.xz | grep 'set 5' | head`
- To track one packet instead, test its line: `(full_addr >> 6) == 0x49d2405`.

Debugging: what you see

```
cpu0_L2C set 5 fill way 6 line 3276c05 ages: 5 2 6 4 1 3 7 0
cpu0_L2C set 5 fill way 7 line 49d2405 ages: 4 1 5 3 0 2 6 7
cpu0_L2C set 5 hit  way 3 line 20d7c05 ages: 3 1 4 7 0 2 5 6
cpu0_L2C set 5 hit  way 0 line 748405 ages: 7 1 3 6 0 2 4 5
```

- The L2C has **8** ways, so ages go from **0** to **7**.
- Each hit or fill moves its way to age **7**, and the lines that were newer age by one.
- After the first fill, way 7 has age **0**: the next fill goes exactly there.
- Remove the print before `make check`: printing slows the run a lot.


SUMS UP MY LIFE
It doesn't work...
But why?
It works!
But why?

How we check your policy

- `make check` builds your policy at the L2C and runs **1M + 5M** instructions on the traces where a wrong policy gives different numbers.
- It compares your **IPC** and your **L2C MPKI** with our solution's. Both must match **exactly**.
- L2C MPKI = L2C misses per 1000 simulated instructions. The makefile computes it for you.
- If either is further off, it prints both your values and ours.

Q: Think like an autograder checking your L2C policy against ours: which of IPC, L2C MPKI and L1D AMAT would you compare, in what combination, and why?

Q: Suppose the autograder only checks "IPC within 5% of ours". At which level (L1D, L2C or LLC) would that still catch a wrong policy, and why?

How good is a policy?

- Checking that a policy is correct is one question. Judging whether it is good is another.
- Say two correct policies give different numbers at the L2C.

Q: Which numbers do you look at to decide which policy is better, and in what order: MPKI, miss rate, or AMAT / average miss latency?

Q: (*) How will the LLC statistics look when you change only the L2C policy?


HOW DO YOU KNOW YOU IMPLEMENTED THE LLC POLICY CORRECTLY?
THE IPC MATCHES.
imgflip.com

It's inlab (3) time

Task 3: replacement by instruction

- Each line was brought in by one instruction, identified by its `ip`.
- Some instructions bring in lines that the program reuses. Others do not.
- We keep a counter per instruction: +1 on a hit, −1 on a miss.
- We evict the line whose instruction has the lowest counter.

Q: How will you store this data?

Q: Where will you update it?

Q: Where will you use it to choose a victim?

H: A line remembers the instruction that brought it in: `current_set[i].ip`.

H: Hits and fills both arrive in `update_replacement_state`; the `hit` argument tells them apart.

Summary

- ChampSim replays a trace on a configured core and caches, and reports IPC and cache statistics.
- A miss travels down through the RQ and MSHR of each level; the data comes back up and fills each level.
- Stores read the line first (RFO), mark it dirty, and write it back when it is evicted.
- Miss rate, MPKI and miss latency measure a cache from different angles.
- A replacement policy decides insertion, promotion and eviction. LRU evicts the least recently used line.
- In `cache.cc`, `handle_read`, `readlike_hit`, `readlike_miss` and `handle_fill` implement the read path.

Lab exam

Part X

Extra: bypassing

- What it is
- When it helps
- In ChampSim

Bypassing

- Some lines are used once and never again, for example an array that the program reads from start to end.
- Storing such a line evicts a line that might have been reused.
- So a policy may **bypass** the cache: the data goes up to the level above, but this cache keeps no copy.
- In ChampSim, `find_victim` bypasses by returning `NUM_WAY` instead of a way.
- ChampSim allows this only at the LLC, and never for a `WRITEBACK`, which must be stored somewhere.

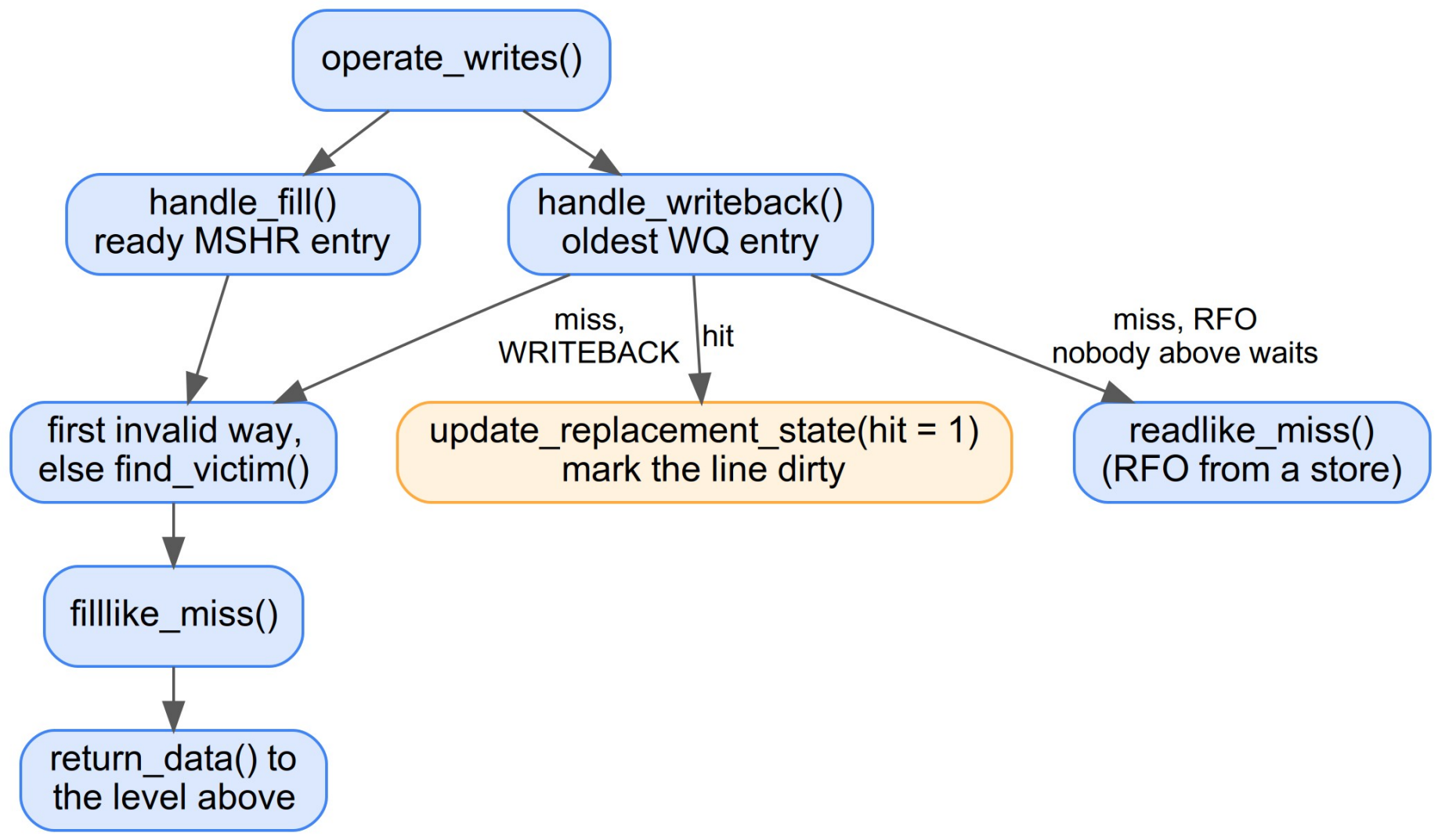
Q: Which of our traces would benefit from bypassing, and how could a policy detect such lines?

Part R

Reference: the write path (view later)

- operate_writes()
- handle_writeback()
- Why a filled line is dirty

The write path (REFERENCE, view later)



handle_writeback (REFERENCE, view later)

```
void CACHE::handle_writeback() {
  while (writes_available_this_cycle > 0) {
    if (!WQ.has_ready()) return;
    PACKET& handle_pkt = WQ.front();             // oldest store or writeback

    uint32_t set = get_set(handle_pkt.address);
    uint32_t way = get_way(handle_pkt.address, set);

    if (way < NUM_WAY) {                         // HIT
      impl_replacement_update_state(..., set, way, ..., /*hit*/ 1);
      block[set * NUM_WAY + way].dirty = 1;      // the line now holds new data
    } else if (handle_pkt.type == RFO && handle_pkt.to_return.empty()) {
      if (!readlike_miss(handle_pkt)) return;    // store miss: read the line first
    } else {
      way = first invalid way, else find_victim(...);
      if (!filllike_miss(set, way, handle_pkt)) return;   // writeback miss: store it
    }
    WQ.pop_front();
    writes_available_this_cycle--;
  }
}
```

handle_writeback, step by step (REFERENCE, view later)

- **Hit**: we tell the policy about the hit and mark the line dirty.
- **Store miss** (an `RFO` whose `to_return` is empty): the store came from the core at this level, so we read the line first through `readlike_miss`.
- **Writeback miss**: the level above sent a dirty line we do not have. We pick a way and store it with `filllike_miss`.
- Like `handle_read`, it stops when the per-cycle limit `max_write` is used up.

Why a filled line is dirty (REFERENCE, view later)

- `fill_block.dirty = (type == WRITEBACK || (type == RFO && to_return.empty()));`
- A **WRITEBACK** fill stores data that is newer than the level below, so the line is dirty.
- An **RFO** with an empty `to_return` started at this level from a store. The store writes into the line, so it is dirty.
- An RFO from the level above has a non-empty `to_return`. The store happens up there, so this copy stays clean.